



11PLANNER

Test Plan
03/24/2026

Suganeshwara Anand
Tamanna Gaur
Ayham Huq
Aaraiz Sohail

Table of Contents

1. Introduction 4
1.1 Purpose 4
1.2 Objective 4
1.3 Scope 4
1.4 Audience 4
2. Test Objectives 4
2.1 Validating Functional Requirements 5
2.2 Verifying System Performance 5
2.3 Ensuring Security 5
2.4 Reliability Testing 5
3. Test Strategy 6
3.1 Testing Approach 6
3.2 Testing Levels 6
3.2.1 Unit Testing 6
3.2.2 Integration Testing 6
3.2.3 System Testing 6
3.2.4 Acceptance Testing 7
3.3 Special Techniques and Tools 7
4. Test Scope 7
4.1 In Scope 7
4.2 Out of Scope 8
4.3 Features for Scope 9
5. Test Deliverables 9
5.1 Functional Test Cases 9
5.2 Non-Functional Test Cases 24
5.3 Integration Test Cases 25
5.4 System Test Cases 32
5.5 Acceptance Test Cases 36
5.6 Test Report 39
6. Test Schedule 40
7. Test Environment 41
7.1 Hardware Requirements 41
7.2 Software & Dependency Requirements 41
7.3 Environmental Conditions 42

8. Test Entry and Exit Criteria 43
8.1 Entry Criteria 43
8.2 Exit Criteria 43
9. Test Pass and Fail Criteria 44
9.1 Test Pass Criteria 44
9.2 Test Fail Criteria 44
10. Test Suspension and Resumption Criteria 44
10.1 Suspension Criteria 44
10.2 Suspension Process 45
10.3 Resumption Criteria 45
10.4 Resumption Process 45
11. Test Design and Execution 46
11.1 Approach Selection: Hybrid Methodology 46
11.2 Test Case Structure 46
11.3 Test Execution Plan 46
12. Test Data & Defect Management 47
12.1 Test Data Management 47
12.2 Defect Management Process 47
12.2.1 Defect Logging 47
12.2.2 Defect Triage and Prioritization 48
12.2.3 Defect Resolution Lifecycle 48
13. Risk Analysis 48
13.1 Technical Risks 48
13.2 Environmental Risks 49
13.3 Human Risks 49
13.4 External Factors 49
13.5 Categorized Mitigation Execution Strategies 49
13.5.1. Security Risk Mitigation 49
13.5.2. Performance Risk Mitigation 49
13.5.3. Dependency Risk Mitigation 50
13.5.4. Data Integrity Risk Mitigation 50
14. Roles and Responsibilities 50

1. Introduction

1.1 Purpose

The purpose of this document is to establish a clear and organized approach to evaluating the 11Planner application. By defining structured testing procedures, the development team can systematically assess how well the software meets its intended design and requirements.
These testing phases are essential for identifying defects (such as inaccuracies in the syllabus parser or user interface issues within the dashboard) so they can be resolved before final release. Ultimately, this ensures a reliable and smooth user experience for students.

1.2 Objective

The objective of this test plan is to thoroughly evaluate the functionality, performance, and security of the 11Planner platform. Testing will focus on validating that all core features operate as expected, including:

- Accurate generation of weekly schedules
- Reliable extraction of course deadlines from syllabus documents
- Seamless interaction between frontend and backend components

In addition, non-functional requirements will be assessed to ensure efficiency and usability. For example, resource-intensive features such as the Focus Dashboard's live media backgrounds will be tested to confirm they run smoothly without negatively impacting device performance.

1.3 Scope

The scope of testing includes all proprietary components developed by the 11Planner team. This includes:

- Next.js frontend user interfaces
- FastAPI backend services and endpoints
- Custom Python scheduling algorithms

Third-party tools and external dependencies are excluded from direct testing. Technologies such as Next.js, Tailwind CSS, external AI models used for text extraction, and browser APIs are assumed to be stable and maintained by their respective providers.
If a defect originates solely from an external dependency rather than the project's implementation, resolving that issue will be considered outside the scope of this project.

1.4 Audience

This test plan is intended for:

- Developers, who will use it to understand quality expectations and validation requirements
- Quality assurance testers, who will follow the defined procedures to execute tests
- Project stakeholders and academic reviewers, who will use this document to evaluate the thoroughness and professionalism of the testing process

Readers are expected to have a basic understanding of software development and testing concepts, though no prior familiarity with the 11Planner codebase is required.

2. Test Objectives

The fundamental goal of our testing phase is to prove that 11Planner genuinely simplifies a student's academic life through a stable, fast, and secure platform. To achieve this, our testing efforts are categorized into four primary objectives:

2.1 Validating Functional Requirements

Our first priority is to confirm that the core features of 11Planner behave exactly as intended. We must validate the entire user journey, from initial data entry to final schedule execution. Specific functional objectives include:

- User Workflows: Verify all core user user actions, including creating tasks, managing courses, and interacting with the UI.
- Database Interactions: Validate all data CRUD operations (Create, Read, Update, Delete) for tasks and calendar blocks.
- Frontend/Backend Integration: Test the integration between the Next.js frontend interfaces and the FastAPI backend routes.
- Third-Party Integrations: Verify Canvas assignment import functionality operates correctly.
- Core Algorithms: Validate the schedule generation algorithm, ensuring the AI optimization correctly allocates time without double-booking.
- Error Handling: Ensure the system handles bad inputs properly and displays user-friendly errors without exposing sensitive backend data or stack traces.

2.2 Verifying System Performance

Because 11Planner integrates complex backend AI calculations with an interactive frontend, performance validation is critical. We want to ensure the application feels snappy, even on older student hardware. Our performance objectives include:

- API Response Times: Verify response times for standard API endpoints, targeting less than 500ms for routine requests.
- Stress Testing: Test system behavior under load, specifically observing how the backend handles concurrent schedule generation requests.
- Query Optimization: Validate database query performance, taking advantage of our in-memory data structures where applicable.
- Algorithmic Speed: Ensure the schedule generation computational response time is under 5 seconds for calculating 10 tasks.
- Frontend Loading: Ensure standard Next.js page load times remain under 3 seconds, including the rendering of the media-heavy Focus Dashboard.

2.3 Ensuring Security

Students rely on their planners to keep their academic schedules and personal information secure. Our objectives for system security guarantee a protected environment:

- Authentication & Authorization: Thoroughly test user login, access controls, and session management.
- Input Defense: Validate input sanitization on all forms and API endpoints to prevent Cross-Site Scripting (XSS) and database injection attacks.
- API Key Protection: Verify that our OpenAI API key is properly secured via environment variables and is never exposed in logs, network traffic, frontend payloads, or source code.
- Credential Protection: Validate password security, ensuring all credentials utilize proper hashing algorithms and are never stored in plain text.
- Data Encryption: Ensure that all data transferred between the client and server is encrypted in transit using HTTPS/TLS.

2.4 Reliability Testing

The system must be highly reliable; students cannot afford to lose their schedule right before an exam. We need to verify that 11Planner remains stable even under adverse conditions:

- Fallback Behaviors: Test system behavior and application state when third-party APIs (Canvas, OpenAI) are rate-limited or entirely unavailable.
- Network Resilience: Verify graceful frontend error handling for sudden network failures or dropped Wi-Fi connections.
- State Persistence: Validate data persistence across application restarts, including local storage preferences for the Focus Dashboard.
- Session Management: Test session timeout handling to ensure users are safely logged out after prolonged inactivity.
- Disaster Recovery: Verify that in the event of an application error or crash, the system recovers smoothly without permanent data loss.

3. Test Strategy

3.1 Testing Approach

For the 11Planner project, we will employ a Combination Strategy (Gray-box Testing), merging the strengths of both Black-box and White-box methodologies to maximize test coverage while maintaining realistic user scenarios:

- **Black-box Testing:** We will test user-facing features and API contracts purely based on input and output, without regarding internal code implementation. This ensures workflows like authentication, UI interactions, and standard task management function correctly for the end user.
- **White-box Testing:** Our developers will test code logic, strict error handling, and sensitive integration points directly at the source level. This enables us to rigorously validate the internal mathematics of the schedule algorithm and the specific schemas of our database interactions.
- **Combination Execution:** By combining both approaches, our team can verify the technical perfection of the backend algorithms while simultaneously ensuring the frontend remains intuitive and stable.

3.2 Testing Levels

3.2.1 Unit Testing

Scope: Individual functions, classes, methods, and React components tested in complete isolation.
Examples:

- The predictive Python schedule generator algorithm (schedule_generator.py).
- Frontend TypeScript date parsing utility functions (dateUtils.ts).
- Course code string-parsing logic and task creation form validation.

Tools: The pytest framework for Python (Backend) and the Jest framework for JavaScript/TypeScript (Frontend).
Coverage Target: Minimum 85% code coverage for all backend logic.

3.2.2 Integration Testing

Scope: The verification of interactions between distinct system components, databases, and third-party APIs.
Examples:

- Data serialization between the Next.js frontend and the FastAPI backend.
- Communication and formatting with the OpenAI API for syllabus parsing.
- Database connection, querying, and mutation operations with Supabase.
- External data synchronization protocols interfacing with the Canvas API.

Tools: Postman for API endpoint contract testing and Python's unittest.mock for simulating and mocking required external service payloads.

3.2.3 System Testing

Scope: Evaluating full end-to-end user workflows alongside critical non-functional system requirements.

Functional Examples:

- A complete user operational journey: *Login → Import Courses → Create Tasks → Generate Schedule → View/Edit Schedule*.
- Stress-testing schedule generation by supplying multiple simultaneous tasks with conflicting priority constraints.

Non-Functional Focus:

- Performance: Routine API response time and UI render validation.
- Security: Validating authentication tokens, Supabase Row Level Security (RLS), and API key obfuscation.
- Reliability: Forcing network drops to verify frontend data persistence and backend error recovery without data loss.

3.2.4 Acceptance Testing

Scope: Validating the software against final user acceptance criteria to guarantee project objectives are met.
Examples:

- Confirming the AI parser and schedule generation produce logically valid, highly usable outputs.
- Verifying the resulting schedules strictly respect user-defined boundary preferences, such as working hours, blocked days, and maximum daily study limits.
- Ensuring an actual student can theoretically plan an entire academic semester using only the software.

Approval: Formal sign-off and written confirmation based on project rubrics provided by the Graduate Teaching Assistant (GTA) or Professor over the course of the semester.

3.3 Special Techniques and Tools

Technique	Tool	Application
API Testing	Postman / cURL	To test FastAPI endpoint request/response structures.
Frontend Testing	Jest + React Testing Library	For executing isolated component behavior tests in the Next.js UI.
End-to-End Testing	Playwright / Cypress	Automated browser testing to simulate real-user mouse clicks and navigation.
Performance Testing	k6 or Artillery	For simulating heavy user loads and stress-testing the FastAPI backend.
Mock Services	unittest.mock (Python)	To isolate system logic by returning simulated Canvas and OpenAI network responses, preventing API token waste.
Code Quality	ESLint, Pylint	Static analysis to ensure syntax consistency and coding standards are maintained.
CI/CD Automation	GitHub Actions	Automatically triggers the test suites on every pull request to prevent breaking changes.

4. Test Scope

4.1 In Scope

Our testing process will rigorously validate the following features and user workflows across the 11Planner application, covering both frontend interactions and backend logic:

1. **User Authentication & Account Management**

- User registration and initial signup flow.
- Secure login processes using email and password credentials.
- Password reset workflows via email link.
- Persistent session management and state authorization.

2. **Course Management**
    - Standard CRUD operations (Create, Read, Update, Delete) for academic courses.
    - Adding, saving, and verifying specific course meeting times and days.
    - Reliable data rendering of the ongoing user course list on the frontend UI.
3. **Task/Assignment Management**
    - Manual creation, form validation, and data entry for tasks.
    - Automated, secure task importation via the Canvas API.
    - Editing task details (modifying title, strict deadlines, estimated hours, and priority weighting).
    - Task deletion actions and the functional UI state change for marking tasks as complete.
4. **Schedule Generation**
    - Automated generation of optimal schedule blocks strictly for pending/uncompleted tasks.
    - Integration, accuracy, and logic validation of the AI-powered Python schedule optimization.
    - Algorithm adherence to user constraints (e.g., maximum daily study limits, assigned work hours).
    - Backend stability and response integrity when handling multiple concurrent generation requests.
5. **Schedule Viewing and Management**
    - Accurate visual rendering of the generated schedule within our React calendar UI formats.
    - Manual user adjustments regarding schedule block lengths.
    - Interactive drag-and-drop capability for moving schedule items fluidly.
    - UI components accurately displaying deep schedule details upon clicking a block.
6. **User Preferences**
    - Setting, saving, and retrieving basic work hours.
    - Locking and saving preferred active study workdays.
    - Enforcing maximum, rigid study hour limits per day across the global state.
    - Storing custom work style configurations locally and remotely.
7. **Focus Features (Dashboard)**
    - Background selection parsing and active video display reliability.
    - Ambient audio selection, generation, and uninterrupted playback loops.
    - Frontend component timer UI functionality and alert delivery.
8. **API Integration Validation**
    - Proper data handshakes involving the Canvas assignment synchronization.
    - Outbound prompts packaging to OpenAI for schedule structuring.
    - Secure data querying and JSON mutations involving the Supabase database.

4.2 Out of Scope

We will not dedicate testing resources to variables outside our required functionality, prioritizing our time on core application stability. Explicitly excluded from our testing scope are:

1. **Mobile/Native Applications:** Testing is strictly limited to responsive web and desktop browser environments. Dedicated iOS/Android native testing is excluded.
2. **Third-Party OAuth:** Implementations of external identity providers (e.g., “Login with Google”, “Login with Microsoft”) are excluded from this primary phase.

3. **Internationalization (i18n):** Support for non-English languages, UI translations, and localization metrics will not be formally assessed.
4. **Extreme Performance Thresholds:** Massive enterprise-scale stress testing is excluded. We will only validate that standard baseline metrics (responses <500ms) are maintained under normal loads.
5. **Infrastructure and DevOps:** Server provisioning, cloud deployment pipelines, containerization logic, and baseline server setups are excluded.
6. **Database Migration:** Legacy data handling and large-scale data migration/recovery scripts are excluded.

4.3 Features for Scope

For a high-level executive overview, the core components bounded within this testing phase are summarized as follows:

1. **User Authentication:** Complete signup, login, and password reset capability.
2. **Course Management:** Standard logic for course CRUD operations and native class block schedules.
3. **Task Management:** Synchronization of Canvas imports alongside reliable manual entry and editing.
4. **Schedule Generation:** Successful AI-driven block scheduling and robust manual user overrides.
5. **Schedule Viewing:** Beautiful and exact rendering of weekly, monthly, and custom UI arrays.
6. **Profile Settings:** Core time-management customization and student preference enforcement.
7. **Focus Features:** Performant Dashboard environments, fluid visual transitions, and high-quality audio routing.
8. **Data Import:** Validated payload verification with outside LMS platforms (Canvas).

5. Test Deliverables

5.1 Functional Test Cases

ID	TC- 01
Item to Test & Description	Sign Up for 11Planner: Test that user authentication is successful with connection to the Supabase Authentication API when a user signs up (creates an account)
Pre-Conditions	1. User is logged out of 11Planner. 2. Frontend is running and reachable. 3. Supabase Authentication service is available. 4. Test inbox for 11PlannerTest@gmail.com (password: TestPlanner26!) is accessible. 5. The test email is not currently registered (or was removed before test run).
Test Steps	1. Navigate to the “Register/Login” page. 2. Click the “Register” button on the login form. 3. Enter the following fields: a. First Name: Test b. Last Name: Student c. Email: 11PlannerTest@gmail.com d. Password: Test1! e. Confirm Password: Test1! 4. Click the “Register” button. 5. Verify a registration success message appears (email verification required). 6. Open 11PlannerTest@gmail.com inbox. 7. Locate the Supabase verification email. 8. Open the email and click the verification link. 9. Return to 11Planner and log in using: a. Email: 11PlannerTest@gmail.com b. Password: Test1!

	10. Navigate to a protected page (for example Dashboard).
Expected Results	• Registration request is accepted. • Verification email is delivered to the inbox. • Verification link activates the account. • User can log in after verification. • Protected page is accessible with authenticated session.
Priority	High
Pass/Fail Criteria	Pass: User account is created, verified, login succeeds, and protected pages are accessible. Fail: Any of the following occur: registration error, no verification email, verification fails, login fails, or protected page access is denied.

ID	TC-02
Item to Test & Description	Log Into 11Planner: Test that a login request is successful with valid and existing user credentials existing in the Supabase Authentication API
Pre-Conditions	1. User account 11PlannerTest@gmail.com exists and is verified. 2. User is logged out. 3. Frontend is running. 4. Supabase Authentication service is available.
Test Steps	1. Navigate to the “Register/Login” page. 2. Enter login credentials: a. Email: 11PlannerTest@gmail.com b. Password: Test1! 3. Click the “Log In” button. 4. Confirm app redirects to “Profile” page. 5. Refresh the browser page once. 6. Confirm user remains logged in after refresh. 7. Navigate to another protected page (for example Tasks). 8. Confirm page loads without redirecting to login.
Expected Results	• Login request succeeds and session is created. • Redirect to authenticated area occurs. • Session persists after refresh. • Protected pages remain accessible while logged in.
Priority	High
Pass/Fail Criteria	Pass: Valid credentials create a persistent session and allow protected navigation. Fail: Login fails, session is not created/persisted, or protected pages redirect to login.

ID	TC-03
Item to Test & Description	Create Course: Verify that a valid course is created and shown across course-dependent areas.
Pre-Conditions	1. User is logged in. 2. Frontend is running. 3. Backend is running. 4. Network connection is active. 5. User is on the “Courses” page.
Test Steps	1. Click the “New Course” button. 2. Enter the following fields: a. Course Name: Calculus I b. Course Code: MATH 201 c. Professor Name: Dr. Smith d. Course Days: Monday, Wednesday, Friday e. Start Time: 9:00 AM f. End Time: 10:00 AM

	g. Description: Review integration techniques, sequences, and series 3. Click the “Create Course” button. 4. Confirm success message appears. 5. Confirm new course card appears on Courses page with matching values. 6. Refresh the “Courses” page. 7. Confirm course card remains visible after refresh. 8. Open Task creation flow and verify course appears in “Select Course” dropdown.
Expected Results	• Course is created successfully with all entered values. • Course card appears immediately in Courses page. • Course persists after page refresh. • Course is available in dependent flows (for example Task association).
Priority	High
Pass/Fail Criteria	Pass: Course is created, displayed, persisted, and selectable in related features. Fail: Course creation fails, display is incorrect/missing, data does not persist, or course is unavailable in dependent features.

ID	TC-04
Item to Test & Description	Create Course (Validation): Verify that invalid course input is rejected and clear error messages are shown.
Pre-Conditions	1. The user is logged into 11Planner. 2. The frontend and backend are running. 3. Network connection is active. 4. The user is on the “Courses” page.
Test Steps	1. Click the “New Course” button. 2. Enter the following inputs: a. Course Name: (leave blank) b. Course Code: MATH 202 c. Professor: Dr. Smith d. Course Days: Monday, Wednesday, Friday e. Start Time: abc f. End Time: xyz 3. Click the “Create Course” button. 4. Verify an error message is shown for leaving required fields blank. 5. Replace blank Course Name with the input: Calculus II 6. Click the “Create Course” button again. 7. Verify an error message is shown for invalid time format. 8. Replace invalid times with another invalid case: a. Start Time: 11:00 AM b. End Time: 9:00 AM 9. Click the “Create Course” button again. 10. Verify an error message is shown indicating start/end time conflict. 11. Refresh the “Courses” page. 12. Verify no new course card was created.
Expected Results	• Invalid course inputs prevent course creation. • Invalid course inputs display specific, related validation errors. • Invalid course does not get stored in the Supabase database.
Priority	Medium
Pass/Fail Criteria	Pass: Proper error messages appear for both invalid formats and invalid time order, and no course is saved. Fail: Invalid course data is accepted, no useful error is shown, or a course is created despite invalid fields.

ID	TC- 05
Item to Test & Description	Test that when a task is created that it is stored within the Supabase Database
Pre-Conditions	1. The user is logged into 11Planner. 2. The frontend and backend are running. 3. Network connection is active. 4. At least one course already exists for task association. 5. The user is on the “Tasks” page.
Test Steps	1. Click the “New Task” button. 2. Enter the following inputs: a. Task Title: Chapter 4 Practice Problems b. Course: MATH 201 c. Deadline: Tomorrow, 8:00 PM d. Estimated Hours: 2 e. Priority: High f. Description: Solve odd problems 1-25 3. Click the “Create Task” submit button. 4. Verify a success message is shown. 5. Verify the task appears in the active/in-progress task list. 6. Refresh the “Tasks” page. 7. Verify the created task remains visible after refresh. 8. Navigate to “Schedule” and verify this task is available for scheduling/generation workflows.
Expected Results	• Task is created successfully with all entered values. • Task is immediately shown in the task list. • Task persists after page refresh. • Task is available in dependent workflows (e.g. Schedule Generation).
Priority	High
Pass/Fail Criteria	Pass: Task is created successfully, displayed correctly, retained after refresh, and usable in dependent flows. Fail: Task is not created, disappears after refresh, or is unavailable for schedule-related features.

ID	TC- 06
Item to Test & Description	Create Task (Validation): Verify that invalid task input is rejected and task is not created.
Pre-Conditions	1. The user is logged into 11Planner. 2. The frontend and backend are running. 3. Network connection is active. 4. The user is on the “Tasks” page.
Test Steps	1. Click the “Add Task” or “New Task” button. 2. Enter invalid/incomplete values: a. Task Title: (leave blank) b. Course: (leave unselected) c. Deadline: Yesterday, 5:00 PM d. Estimated Hours: -2 e. Priority: Medium 3. Click the “Create Task” submit button. 4. Verify error messages are shown for missing required fields. 5. Replace blank values with the following inputs: a. Task Title: Chapter 3 Practice Problems b. Course: MATH 201 6. Click the “Create Task” submit button. 7. Verify error message is shown for invalid estimated hours.

	8. Replace invalid Estimated Hours with the input: 2 9. Click the “Create Task” submit button. 10. Verify error message is shown for past deadline. 11. Refresh the “Tasks” page. 12. Verify no new task was created.
Expected Results	• Invalid task inputs prevent task creation. • Invalid task inputs display specific, related validation errors. • Error messages are displayed for invalid fields • Invalid task does not get stored in the Supabase database.
Priority	Medium
Pass/Fail Criteria	Pass: Submission is blocked, all relevant validation errors are shown, and no task is created. Task is saved in Supabase database Fail: Invalid task is accepted, no clear error appears, or task is created despite invalid inputs. Task is saved in Supabase database

ID	TC-07
Item to Test & Description	Generate Schedule: Verify that schedule generation uses existing courses and tasks to create study blocks, displays them in the schedule UI, and stores them in the backend.
Pre-Conditions	1. User is logged into 11Planner with a valid account. 2. User has exactly these courses saved: a. Course A: Meets Monday, Wednesday, Friday from 09:00 AM to 10:00 AM. b. Course B: Meets Tuesday, Thursday from 01:00 PM to 02:15 PM. 3. User has exactly these active tasks saved: a. Task 1: Due in 2 days, Estimated Hours = 3, Priority = Critical. b. Task 2: Due in 4 days, Estimated Hours = 2, Priority = Medium. c. Task 3: Due in 7 days, Estimated Hours = 4, Priority = Low. 4. Preference Quiz has exactly these preferences saved: a. Preferred Work Time = Afternoon. b. Preferred Work Days = every day selected. c. Break Timings = Classic Pomodoro. d. Assignment Plan = Balanced. 5. Daily Study Limit in Settings is set to 4 hours/day and saved successfully. 6. Frontend, backend, and Supabase services are running with no active error banners.
Test Steps	1. Navigate to the “Schedule” page. 2. Select Week view. 3. Click the “Generate Schedule” button exactly once. 4. Verify that new study blocks appear in the Week view calendar grid. 5. Verify that new study blocks are generated between 12:00 PM to 6:00 PM. 6. Add up the total hours of work per day of the generated study blocks. 7. Verify that the total hours for each day are less than or equal to 4 hours. 8. Open browser DevTools -> Network tab. 9. Find the schedule generation request. 10. Verify the request response status is HTTP 200. 11. Verify the response body contains a non-empty schedule payload (at least one generated block). 12. Open Supabase table for schedule_blocks for the current user and current week. 13. Verify new rows were inserted after generation time.

	14. Return to the app and refresh the Schedule page once. 15. Verify generated blocks are still visible after refresh.
Expected Results	• Week view displays generated study blocks for the current user. • Schedule generation API call returns HTTP 200. • API response contains a non-empty schedule payload. • Supabase schedule table contains newly inserted rows for the current user/week. • Generated blocks remain visible after page refresh. • No UI error message appears.
Priority	High
Pass/Fail Criteria	Pass: Schedule blocks are generated and visible in the "Schedule" page. API call returns HTTP 200 with non-empty payload. New schedule rows are stored in Supabase for the correct user and week. Blocks persist after refresh and no UI errors appear. Fail: No schedule blocks appear, or generation fails. API call is non-200 or payload is empty. No new rows are stored in Supabase. Blocks disappear after refresh or UI shows generation/storage error.

ID	TC-08
Item to Test & Description	Task Prioritization in Schedule Generation: Verify that higher-priority tasks with closer deadlines are scheduled before lower-priority tasks with later deadlines when all other constraints are equal.
Pre-Conditions	1. User is logged into 11Planner with a valid account. 2. User has at least one course with fixed class time to create realistic constraints (for example, Monday/Wednesday/Friday 09:00 AM-10:00 AM). 3. User has no manually locked schedule blocks for the current week. 4. User has exactly these active tasks saved before generation: a. Task A: Title = "Urgent Quiz Prep", Due = tomorrow at 06:00 PM, Estimated Hours = 2, Priority = Critical. b. Task B: Title = "Reading Notes", Due = 7 days from now at 06:00 PM, Estimated Hours = 2, Priority = Very Low. c. Task C: Title = "Problem Set Draft", Due = 3 days from now at 06:00 PM, Estimated Hours = 2, Priority = Medium. 5. Preference Quiz has exactly these preferences saved: a. Preferred Work Time = Afternoon. b. Preferred Work Days = every day selected. c. Break Timings = Classic Pomodoro. d. Assignment Plan = Balanced. 6. Daily Study Limit in Settings is set to 2 hours/day. 7. Frontend, backend, and Supabase are running with no active errors.
Test Steps	1. Navigate to "Schedule" page and select Week view. 2. Click the "Generate Schedule" button once. 3. Identify the first 3 generated study blocks in chronological order (earliest date/time to latest). 4. Record which task each of those first 3 blocks belongs to. 5. Open browser DevTools -> Network tab. 6. Open the schedule generation response and verify task assignments in the earliest returned blocks.

	7. Confirm that blocks assigned to Task A appear before blocks assigned to Task C and Task B whenever free time is available. 8. Confirm Task C blocks appear before Task B blocks in early time slots when no conflict prevents it. 9. Refresh Schedule page and confirm ordering remains consistent.
Expected Results	• Earliest generated study blocks are allocated to Task A first (highest urgency + highest priority). • Task C is scheduled before Task B in earlier available slots. • Ordering in UI matches API response ordering/assignment intent. • No schedule generation error message appears. • Scheduling order is stored within Supabase
Priority	High
Pass/Fail Criteria	Pass: Generated ordering reflects urgency/priority logic: Task A first, then Task C, then Task B under comparable constraints. UI block assignments and API payload are consistent. Fail: Lower-priority/later-due tasks are scheduled before higher-priority/earlier-due tasks without valid conflict reason. UI and API assignment ordering are inconsistent. Generation fails or errors are shown.

ID	TC-09
Item to Test & Description	Preference and Study Limit Enforcement: Verify schedule generation respects daily study-hour cap and preference quiz work-time constraints.
Pre-Conditions	1. User is logged in. 2. User has at least 1 course and at least 4 active tasks totaling at least 8 estimated hours due within the same week. 3. Settings are editable and Preference Quiz is accessible. 4. No old schedule is present which would prevent schedule creation 5. Frontend, backend, and Supabase are running.
Test Steps	1. Navigate to “Settings” page. 2. Set Study Limit slider to 2 hours/day. 3. Save settings and verify a success confirmation appears. 4. Navigate to “Profile” page and select the Preference Quiz. 5. Set preferences to: a. Preferred Work Time = Evening. b. Preferred Work Days = Monday-Friday. c. Break Timings = Deep Work. d. Assignment Plan = Balanced. 6. Submit and save quiz responses. 7. Navigate to Schedule page and select Week view. 8. Click Generate Schedule once. 9. For each weekday (Monday-Friday), sum the total generated study-block duration. 10. Confirm each weekday total is less than or equal to 2 hours. 11. Confirm generated block start times are primarily in evening windows based on the preference quiz 12. Inspect generated block lengths and confirm they are primarily 45-60 minutes long based on preference quiz. 13. Open Network tab and verify generation request completed with HTTP 200. 14. Refresh Schedule page and confirm generated blocks persist.

Expected Results	• Generated schedule does not exceed 2 hours of study blocks per day (for configured days). • Block timing reflects saved worktime preference (evening-focused placement). • Block lengths correspond to saved focus timing preference (deep work). • Schedule generation API call succeeds with HTTP 200. • Generated schedule persists after refresh.
Priority	High
Pass/Fail Criteria	Pass: Daily study cap is enforced across the generated week. Worktime preference is applied to block placement Generated block sessions match with break timing preference. Generation is successful without errors Fail: Any day exceeds configured 2-hour cap without system message Any block's duration goes over 60 minutes. Preferences are ignored with no warning/fallback message. Generation fails, returns non-200, or data does not persist.

ID	TC-10
Item to Test & Description	Course Conflict Prevention: Verify generated study blocks do not overlap with fixed course meeting times.
Pre-Conditions	1. User is logged in. 2. User has these course times saved: a. Course A: Monday/Wednesday/Friday 09:00 AM-10:00 AM. b. Course B: Tuesday/Thursday 01:00 PM-02:15 PM. 3. User has enough active tasks (minimum total 6 estimated hours) to create multiple study blocks. 4. Preference Quiz has exactly these preferences saved: a. Preferred Work Time = Morning. b. Preferred Work Days = every day selected. c. Break Timings = Classic Pomodoro. d. Assignment Plan = Balanced. 5. Frontend, backend, and Supabase are running.
Test Steps	1. Navigate to "Schedule" page and select Week view. 2. Click the "Generate Schedule" button once. 3. Verify that Monday, Wednesday, and Friday between 09:00 AM and 10:00 AM has no generated study block which overlaps these intervals. 4. Open Day view for one M/W/F day to confirm no overlap in the detailed view. 5. Open generation response in Network tab and confirm returned blocks do not contain intervals overlapping class times. 6. Refresh page and re-check one M/W/F day. 7. Navigate to "Profile" page and select the Preference Quiz. 8. Change Preferred Work Time to Afternoon. 9. Navigate to "Schedule" page and select Week view. 10. Click the "Generate Schedule" button again. 11. Inspect Tuesday and Thursday between 01:00 PM and 02:15 PM. 12. Verify no generated study block overlaps these intervals. 13. Open Day view for one T/Th day to confirm no hidden overlap in detailed view. 14. Open generation response in Network tab and confirm returned blocks do not contain intervals overlapping class times.

	15. Refresh page and re-check one T/Th day.
Expected Results	• Generated study blocks avoid all fixed class intervals. • No overlapping entries appear in calendar UI at class times. • Schedule remains consistent with regards to course timings after refreshing.
Priority	High
Pass/Fail Criteria	Pass: No overlaps exist between generated study blocks and course meeting times in UI Schedule remains consistent after refresh. Fail: Any generated block overlaps a class interval. Conflict appears after refreshing the page

ID	TC-11
Item to Test & Description	Break Preference Enforcement: Verify schedule regeneration applies break-related preference quiz settings and updates study block structure accordingly.
Pre-Conditions	1. User is logged into 11Planner. 2. Frontend, backend, and Supabase services are running 3. User has at least 1 course and at least 3 active tasks with total estimated hours greater than or equal to 6. 4. A previously generated schedule exists for the current week (baseline schedule).
Test Steps	1. Navigate to Schedule page and record the generated schedule pattern (number of blocks and their spacing). 2. Navigate to the “Profile” page and select the Preference Quiz. 3. Set break-related preferences to configuration A: a. Break Timings = Short Bursts. b. Assignment Plan = Balanced. 4. Save quiz responses and confirm success message. 5. Navigate back to “Schedule” page. 6. Click “Generate Schedule” once. 7. Record resulting block pattern (block lengths and spacing/gaps between blocks). 8. Navigate again to the “Profile” page and select the Preference Quiz. 9. Change break-related preferences to configuration B: a. Break Timings = Full Immersion. b. Assignment Plan: One Big Session. 10. Save quiz responses and confirm success message. 11. Return to “Schedule” page and click “Generate Schedule” once. 12. Compare regenerated pattern (configuration B) against previous pattern (configuration A). 13. Verify schedule changed in a way that matches break preferences
Expected Results	• Regeneration applies saved preference changes from the quiz. • Configuration A produces more spacing between sessions than configuration B • No error messages occur
Priority	Medium
Pass/Fail Criteria	Pass: Schedule structure updates after each preference change and regeneration. Block spacing/session structure is in line with the selected break/focus preferences. Regeneration succeeds with no errors.

	Fail: Schedule does not change with changed preference settings Break-related settings do not change Regeneration fails

ID	TC-12
Item to Test & Description	Edit Task Propagation: Verify editing an existing task updates the Tasks page, schedule generation behavior, and backend record.
Pre-Conditions	1. User is logged into 11Planner. 2. Frontend, backend, and Supabase are running 3. "Chapter 4 Practice Problems" task from TC-05 is present on "Tasks" page. 4. At least one other task and at least one course exist (task must remain associated to valid course). 5. A generated schedule exists so task edit impact can be observed after regeneration.
Test Steps	1. Navigate to "Tasks" page. 2. Click "Edit" for the task titled "Chapter 4 Practice Problems". 3. Update the task fields as follows: a. Title: change to "Updated Task - Integration Review". b. Deadline: change to 2 days from current date at 08:00 PM. c. Estimated Hours: change to 3. d. Priority: change to Critical. e. Description: add "Focus on chapters 5 and 6". 4. Click "Update Task". 5. Verify success message appears. 6. Confirm updated values appear in the task row/card on "Tasks" page. 7. Refresh "Tasks" page and confirm edited values persist. 8. Navigate to "Schedule" page and click "Generate Schedule". 9. Verify updated task appears in generated study blocks with timing/ordering that match with new urgency/priority. 10. Verify that the record is in Supabase Tasks backend table
Expected Results	• Task edits save successfully and are shown on the Tasks page. • Edited task data persists after refresh. • Regenerated schedule uses updated task fields (especially due date/priority/estimated hours). • Supabase task row is updated with new information
Priority	Medium
Pass/Fail Criteria	Pass: All edited task fields are saved and displayed correctly in UI. Regenerated schedule reflects edited task constraints/priority. Backend data matches edited values after refresh. Fail: Edits are not saved Schedule does not factor in edited task values. Backend Supabase row is unchanged or update returns error.

ID	TC-13
Item to Test & Description	Delete Task Propagation: Verify deleting an existing task removes it from Tasks page, generated schedule, and backend storage.
Pre-Conditions	1. User is logged in. 2. Frontend, backend, and Supabase are running.

	3. "Chapter 4 Practice Problems" task from TC-05 is present on "Tasks" page. 4. Generated schedule currently includes blocks related to that task.
Test Steps	1. Navigate to "Tasks" page. 2. Click "Delete" for the task titled "Chapter 4 Practice Problems". 3. Confirm deletion in the confirmation dialog. 4. Verify success message appears. 5. Confirm task is removed from active task list immediately. 6. Refresh "Tasks" page and confirm task remains removed. 7. Navigate to "Schedule" page and click "Generate Schedule". 8. Verify no study blocks are generated for the deleted task. 9. Verify dashboard/stat cards update accordingly (for example total active tasks decreases by one). 10. Verify backend row in Supabase Tasks table is removed for current user.
Expected Results	• Deleted task is removed from Tasks UI and stays removed after refresh. • Regenerated schedule does not show time blocks for the deleted tasks. • Dependent items such as dashboard counts show deletion • Task row is removed from Supabase.
Priority	Medium
Pass/Fail Criteria	Pass: Task is fully removed from UI, backend, and schedule generation output. Related counts/metrics update correctly after deletion. Fail: Task remains visible in UI or reappears after refresh. Schedule still shows deleted task blocks. Supabase data row is not deleted or error message is shown

ID	TC-14
Item to Test & Description	Edit Course Propagation: Verify editing an existing course updates Courses page, Schedule views, and backend records consistently.
Pre-Conditions	1. User is logged in. 2. Frontend, backend, and Supabase are running. 3. "Calculus I" course from TC-03 is present and currently visible in Courses page. 4. At least one other course, one task, and a generated schedule already exist
Test Steps	1. Navigate to "Courses" page. 2. Click "Edit" for the course titled "Calculus I". 3. Update course fields as follows: a. Course Name: change to "Calculus I - Updated". b. Course Days: modify to include Tuesday. c. Start Time: change from 09:00 AM to 10:30 AM. d. End Time: change from 10:00 AM to 11:30 AM. 4. Click "Update Course". 5. Confirm success message appears. 6. Verify updated values appear on the course card in "Courses" page. 7. Navigate to "Schedule" page (Week view). 8. Verify class-time display factors in updated course timing. 9. Click "Generate Schedule". 10. Verify generated study blocks now avoid the updated class timings (10:30 AM-11:30 AM). 11. Refresh page to verify course values stay updated. 12. Verify backend data (Supabase Courses row) matches edited values.

Expected Results	• Course edit is saved successfully. • Courses page immediately shows updated course details. • Schedule view shows new course times. • Backend record show updated values after refresh.
Priority	Medium
Pass/Fail Criteria	Pass: Edited course values are present across UI and backend. Regenerated schedule uses updated course timing blocks. Changes persist after refresh/reload. Fail: Edited values do not appear in UI or backend. Schedule still uses old class time after regeneration. Data is the same or errors message occurs.

ID	TC-15
Item to Test & Description	Delete Course Propagation: Verify deleting an existing course removes it from Courses page, schedule context, and backend.
Pre-Conditions	1. User is logged in. 2. Frontend, backend, and Supabase are running. 3. "Calculus I" course from TC-03 is present and currently visible in Courses page. 4. "Calculus I" course is currently shown in schedule context (class block or related references).
Test Steps	1. Navigate to "Courses" page. 2. Select "Delete" for the course titled "Calculus I". 3. In confirmation dialog, click Confirm Delete. 4. Verify success message appears. 5. Verify deleted course's card is removed from "Courses" page immediately. 6. Refresh "Courses" page and confirm course is still deleted. 7. Navigate to Task creation/edit and verify deleted course is deleted from the Associated Course dropdown. 8. Navigate to "Schedule" page and click "Generate Schedule". 9. Verify deleted course class block is no longer shown in schedule views. 10. Verify backend data (Supabase Courses table) no longer contains the deleted course row for this user.
Expected Results	• Deleted course is removed from UI immediately and after refresh. • Deleted course is removed from dependent selections (task course dropdown). • Regenerated schedule no longer includes deleted course constraints/blocks. • Backend Supabase data is updated by removing the course
Priority	Medium
Pass/Fail Criteria	Pass: Course is fully removed from frontend displays and dependent pages. Schedule no longer includes deleted course when regenerating. Supabase no longer contains course row for the user. Fail: Course remains visible in UI after deletion. Schedule still includes deleted course. Backend row remains in Supabase or deletion shows an error.

ID	TC-16
Item to Test & Description	Syllabus PDF Parsing: Verify the system accurately parses course information from an uploaded PDF syllabus and autofills course form fields.
Pre-Conditions	1. User is logged in. 2. Frontend and backend are running with active internet connection. 3. Supabase file storage and pypdf parsing library are functional. 4. A valid test syllabus PDF file is available with the following known content: a. Course Name: “Data Structures and Algorithms” b. Course Code: “CS 301” c. Instructor Name: “Dr. Rachel Wilson” d. Meeting Days: Tuesday, Thursday e. Start Time: 10:00 AM f. End Time: 11:30 AM 5. User is on Courses page.
Test Steps	1. Click “New Course” button to open course creation form. 2. Click “Upload Syllabus PDF” button. 3. Select and upload the test syllabus PDF file. 4. Wait until the file upload completes and parsing begins (loading indicator visible). 5. Wait until a completion message is shown. 6. Verify the following form fields are now autofilled with parsed values: a. Course Name field contains “Data Structures and Algorithms”. b. Course Code field contains “CS 301”. c. Professor Name field contains “Dr. Rachel Wilson”. d. Course Days contains Tuesday and Thursday selected. e. Start Time field contains “10:00 AM”. f. End Time field contains “11:30 AM”. 7. Confirm manual edit is still possible (click a field and modify the value). 8. Click “Create Course” to store the parsed data. 9. Refresh “Courses” page and verify the course persists with parsed values. 10. Check backend logs to confirm pypdf extraction completed without errors.
Expected Results	• PDF upload and parsing completes without error. • Course form fields are accurately populated from extracted PDF text. • Autofilled data is user-editable before final save. • Parsed course persists after save and refresh. • No parsing or backend error is displayed.
Priority	Low
Pass/Fail Criteria	Pass: PDF parsing succeeds and extracts all target information accurately. Form fields are autofilled with extracted values matching source PDF. Data is editable and saves successfully. Persisted data matches parsed values after refresh. Fail: PDF upload fails or parsing returns an error message. Form fields remain blank despite valid PDF content. Extracted values are inaccurate or mismatched to PDF source. Data fails to persist after save.

ID	TC-17

Item to Test & Description	Canvas Extension Integration: Verify the Chrome extension accurately retrieves assignments from Canvas course page and creates corresponding tasks in 11Planner.
Pre-Conditions	1. User is logged into 11Planner with a valid account. 2. User is logged into Canvas in the same browser. 3. Frontend, backend, and Supabase are running. 4. 11Planner Chrome extension is installed and enabled. 5. Canvas course page contains exactly 3 visible assignments with the following known details: a. Assignment 1: "Quiz Chapter 3", Due Date: tomorrow at 11:59 PM, Points: 10. b. Assignment 2: "Problem Set 5", Due Date: 3 days from now at 11:59 PM, Points: 25. c. Assignment 3: "Essay Submission", Due Date: 7 days from now at 11:59 PM, Points: 50. 6. No pre-existing 11Planner tasks match these Canvas assignments. 7. Active internet connection is available.
Test Steps	1. Open Canvas in browser and navigate to a course with assignments visible. 2. Confirm all 3 test assignments are displayed on the Canvas page. 3. Click the 11Planner extension icon in the browser toolbar. 4. Click "Sync Canvas Assignments" or similar sync button in the popup. 5. Wait until the extension processes and displays a completion message. 6. Navigate to 11Planner in a new tab (or same tab if supported). 7. Go to the "Tasks" page. 8. Verify all 3 assignments appear as new tasks with: a. Task 1: Title contains "Quiz Chapter 3", Due date matches tomorrow. b. Task 2: Title contains "Problem Set 5", Due date matches 3 days from now. c. Task 3: Title contains "Essay Submission", Due date matches 7 days from now. 9. Click on each task and confirm associated course is correctly populated. 10. Query Supabase Tasks table to verify 3 new rows were inserted for the current user. 11. Refresh Tasks page and confirm tasks remain visible and synced.
Expected Results	• Extension successfully parses Canvas DOM and extracts all visible assignments. • Assignments are converted to 11Planner tasks with matching titles and due dates. • New tasks appear immediately in 11Planner UI after sync. • New rows are created in Supabase for current user. • Tasks persist after page refresh. • No CORS errors or extension failures occur.
Priority	Low
Pass/Fail Criteria	Pass: All Canvas assignments are extracted and converted to 11Planner tasks. Task titles, due dates, and course associations match Canvas source data. New tasks are stored in Supabase and visible after refresh. No extension crashes or error messages appear. Fail: No assignments are retrieved or sync fails silently. Retrieved task data is incomplete or inaccurate. New tasks do not appear in UI or Supabase.

	Extension throws CORS, security, or runtime errors.

ID	TC-18
Item to Test & Description	Timer Start/Pause/Resume/Restart Functionality: Verify timer component responds correctly to all control buttons and maintains accurate countdown.
Pre-Conditions	1. User is logged into 11Planner. 2. Frontend is running and Dashboard is accessible. 3. Timer widget is visible on Dashboard. 4. System clock is accessible for time verification. 5. Browser tab is active and focused.
Test Steps	1. Navigate to “Dashboard” page. 2. Locate the Timer widget on the page. 3. Verify timer displays “25:00” as default (standard Pomodoro). 4. Click the “Start” button. 5. Wait exactly 5 seconds using system clock. 6. Verify timer displays “24:55” (acceptable tolerance ±1 second). 7. Wait another 5 seconds. 8. Verify timer displays “24:50” (acceptable tolerance ±1 second). 9. Click the “Pause” button. 10. Wait 5 seconds while paused. 11. Verify timer value remains at “24:50” (no further countdown during pause). 12. Click the “Resume” button. 13. Wait 5 seconds. 14. Verify timer has decremented to “24:45” (countdown resumes correctly). 15. Click the “Restart” button. 16. Verify timer resets immediately to “25:00”. 17. Click “Start” again and wait 10 seconds. 18. Verify timer shows approximately “24:50”. 19. Allow timer to run until completion or manually stop it. 20. Verify a completion notification/sound is triggered when timer reaches 00:00 (or verify behavior matches app design).
Expected Results	• Timer starts and counts down accurately (±1 second tolerance). • Pause freezes countdown immediately. • Resume continues from paused value without reset. • Restart resets timer to 25:00 without error. • Countdown remains accurate across multiple start/pause/resume cycles. • Notification/alert is triggered at completion if applicable. • No timer drift or value jumping occurs.
Priority	High
Pass/Fail Criteria	Pass: All control buttons (Start/Pause/Resume/Restart) function correctly. Countdown is accurate to within ±1 second. Pause and Resume maintain state correctly across multiple cycles. Completion notification triggers as designed. Fail: Timer does not count down or controls have no effect. Pause or Resume fail; values jump or reset unexpectedly. Restart does not reset to 25:00. Countdown drifts or loses sync with system time across multiple cycles. Completion notification fails to appear.

5.2 Non-Functional Test Cases

ID	TC-19
Item to Test & Description	Authentication and Dashboard Redirect: Verify user can log in with valid credentials and is redirected to protected dashboard with user context displayed.
Pre-Conditions	1. User is logged out of 11Planner. 2. Frontend and backend are running. 3. Supabase Authentication service is operational. 4. Valid test account exists with: a. Email: testuser@wayne.edu b. Password: TestPass123! 5. Active internet connection is available.
Test Steps	1. Navigate to the "Login" page. 2. Enter Email: testuser@wayne.edu 3. Enter Password: TestPass123! 4. Click the "Log In" button. 5. Start a stopwatch or note system time to measure redirect latency. 6. Wait for page load to complete (observe no loading spinner). 7. Verify the current URL has changed from /auth to /profile. 8. Verify a welcome message displays with user name or email visible (for example "Welcome, Test User"). 9. Verify the Navigation bar shows authenticated state (Logout button visible, protected links accessible). 10. Verify the Navigation bar displays user-specific menu options (Profile, Settings, Courses, Tasks, Schedule). 11. Verify browser DevTools shows an active session token (JWT or Supabase session cookie). 12. Click on a protected page link (for example Courses or Tasks) and confirm it loads without redirecting back to login. 13. Stop stopwatch and calculate total redirect time from "Log In" button click to dashboard full load.
Expected Results	• Login credentials are accepted and authentication succeeds. • User is redirected to dashboard/home within 3 seconds. • Dashboard displays welcome message with authenticated user context. • Navigation and sidebar reflect logged-in state with all protected links. • Session token is active and valid. • Protected pages are accessible without re-authentication.
Priority	High
Pass/Fail Criteria	Pass: User logs in successfully and is redirected to dashboard within 3 seconds. Welcome message and user context are displayed. Navigation shows logged-in state and protected links are functional. Session token is valid and persists across navigation. Fail: Login fails with credential error. User is not redirected or remains on login page. Redirect takes longer than 3 seconds. Welcome message or user name is not displayed. Protected pages redirect to login unexpectedly.

ID	TC-20
Item to Test & Description	Data Access Security Isolation: Verify one authenticated user cannot access another user's tasks/courses/schedules through direct API calls or UI manipulation.

Pre-Conditions	1. Active internet connection is available. 2. Frontend, backend, and Supabase are running. 3. Two valid user accounts exist: a. User A: usera_test@wayne.edu b. User B: userb_test@wayne.edu 4. User B has at least 1 task, 1 course, and 1 generated schedule saved. 5. User A has no access permission to User B data by design. 6. Browser DevTools is available for network request inspection.
Test Steps	1. Log in as User B. 2. Navigate to Tasks, Courses, and Schedule pages and record one resource ID from each (task_id, course_id, schedule_id) using Network tab responses. 3. Log out User B. 4. Log in as User A in a new clean session. 5. Confirm User A dashboard does not show User B content in UI. 6. Open DevTools -> Network tab and copy a valid authenticated request from User A session (including auth token/cookie automatically sent by browser). 7. Attempt to access User B task directly by calling the corresponding API endpoint with User B task_id. 8. Attempt to access User B course directly by calling the corresponding API endpoint with User B course_id. 9. Attempt to access User B schedule directly by calling the corresponding API endpoint with User B schedule_id. 10. Attempt to edit or delete one User B resource using User A session. 11. Record HTTP status and response body for each unauthorized attempt. 12. Refresh User A UI and confirm no User B data appears. 13. Log back in as User B and confirm User B data remains unchanged and intact.
Expected Results	• User A can only see User A data in UI and API responses. • Direct API access attempts to User B resources are denied (for example HTTP 401/403 or empty result by policy). • Unauthorized update/delete attempts are blocked and do not modify User B data. • No cross-user data leakage appears in any page or response payload.
Priority	High
Pass/Fail Criteria	Pass: All cross-user read/write attempts are blocked. No User B data is exposed to User A. User B data remains unchanged after unauthorized attempts. Fail: Any User B resource is readable, editable, or deletable by User A. Any cross-user data appears in User A UI/API responses.

5.3 Integration Test Cases

ID	TC-21
Item to Test & Description	Task Creation and Dashboard Update: Verify creating a task stores it in backend and immediately reflects updates in dashboard statistics and pending task list.
Pre-Conditions	1. User is logged into 11Planner. 2. Frontend, backend, and Supabase are running. 3. At least one course already exists for task association. 4. Dashboard shows initial pending task count (baseline value noted). 5. Active internet connection is available.

Test Steps	1. Navigate to the “Dashboard” page. 2. Record the current “Total Tasks” count displayed in statistics (baseline). 3. Click “Add Task” quick action or navigate to “Tasks” page. 4. Click “New Task” button. 5. Fill in the following task fields exactly: a. Title: Complete Project Proposal b. Course: Select existing course from dropdown (for example MATH 201) c. Deadline: March 25, 2026 (2 days from test date) d. Estimated Hours: Critical e. Priority Level: High f. Description: Complete the project proposal document 6. Click “Create Task” submit button. 7. Verify a success message appears. 8. Wait 2-3 seconds for backend sync. 9. Verify the new task appears in the pending task list with matching details. 10. Navigate back to Dashboard via sidebar or navigation. 11. Verify the “Total Tasks” count has incremented by exactly 1. 12. Verify Dashboard shows new task in pending task section. 13. Query Supabase Tasks table for current user and verify new row exists with exact field values. 14. Refresh Dashboard page and confirm task count and pending list remain updated.
Expected Results	• Task is created and accepted without validation errors. • Success message is displayed to user. • New task immediately appears in Tasks page pending list. • Dashboard task count increments by 1. • Dashboard displays task in pending section. • Supabase Tasks table contains new row for current user. • Data persists after page refresh.
Priority	High
Pass/Fail Criteria	Pass: Task is created successfully with all entered details. Task immediately appears in Tasks page and Dashboard pending list. Dashboard statistics (task count) update correctly. Supabase contains new task row for current user. Task data persists after refresh. Fail: Task creation fails or error is displayed. Task does not appear in UI or appears with incorrect details. Dashboard statistics do not update or show wrong count. Task is not stored in Supabase. Data is lost after refresh.

ID	TC-22
Item to Test & Description	Full Schedule Generation Workflow: Verify schedule is generated successfully using combined course times, task deadlines, and user preferences.
Pre-Conditions	1. User is logged into 11Planner. 2. Frontend, backend, and Supabase are running. 3. User has exactly these items configured: a. At least 2 courses with fixed class times.

	b. At least 3 active tasks with deadlines within 1 week and varying priorities. c. Preference Quiz is completed with specific work hours, workdays, and break preferences. d. Study limit is set to 4 hours/day. 4. No existing locked/pinned schedule blocks for the current week. 5. Active internet connection is available.
Test Steps	1. Navigate to "Schedule" page 2. Verify dashboard shows pending tasks count (should be 3+) 3. Review pending tasks and courses listed 4. Verify preferences are displayed (work hours, daily limits, etc.) 5. Click "Generate Schedule" button 6. Observe loading indicator (if present) 7. Wait for schedule generation to complete (max 10 seconds) 8. Verify success message appears 9. Verify schedule blocks appear on weekly calendar view 10. Verify all pending tasks have schedule blocks assigned 11. Confirm schedule respects constraints: a. Course meeting times (no conflicts) b. Task deadlines (scheduled before due date) c. Work hour preferences d. Daily study hour limits 12. Verify break times appear between study blocks
Expected Results	• Schedule generation API call completes successfully (HTTP 200). • Study blocks are generated for all pending tasks. • Blocks are placed before task due dates. • Blocks do not conflict with course meeting times. • Daily study hour limit (4 hours) is respected. • Work-hour preferences are applied to block placement. • Generated schedule is stored in Supabase and persists after refresh. • No generation timeout or error occurs.
Priority	High
Pass/Fail Criteria	Pass: Schedule generation completes within 10 seconds without errors. All pending tasks receive assigned study blocks. No course-time conflicts exist. Daily study hour limit is enforced. Generated schedule is stored and persists after refresh. Fail: Generation timeout or API returns non-200 status. One or more tasks have no assigned blocks. Study blocks overlap course times. Daily hour limit is exceeded. Generated schedule is not stored or disappears after refresh.

ID	TC-23
Item to Test & Description	Schedule Regeneration Accuracy: Verify that task changes and preference changes are both reflected in the regenerated schedule.
Pre-Conditions	1. Active internet connection is available. 2. Backend is running. 3. Frontend is running. 4. User is logged in. 5. User already has:

	a. At least 2 existing courses with meeting times. b. At least 3 existing active tasks. c. A previously generated weekly schedule. 6. Current daily study limit is 4 hours/day.
Test Steps	1. Navigate to “Schedule” page. 2. Record baseline schedule details: a. Total number of study blocks shown. b. Earliest study block time for tomorrow. c. Maximum scheduled study hours on any single day. 3. Navigate to “Tasks” page. 4. Create a new urgent task with the following values: a. Title: 3 page Linguistics Essay b. Course: Select any existing course c. Deadline: Tomorrow, 11:59 PM d. Estimated Hours: 4 e. Priority: High 5. Click “Create Task” and verify success message appears. 6. Return to “Schedule” page. 7. Click “Generate Schedule” once. 8. Wait for regeneration to complete (maximum 10 seconds). 9. Verify urgent task appears as one or more study blocks in the schedule. 10. Verify at least one urgent-task block appears before lower-priority/later-deadline tasks (when time slots are available). 11. Navigate to “Settings” page. 12. Change daily study limit from 4 to 6 hours/day. 13. Save settings and verify success message appears. 14. Return to “Schedule” page. 15. Click “Generate Schedule” again. 16. Verify regenerated schedule now allows up to 6 study hours in a day. 17. Verify previously scheduled blocks are adjusted to reflect both: a. New urgent task insertion. b. New 6-hour daily cap. 18. Refresh “Schedule” page and verify regenerated blocks persist.
Expected Results	• Schedule regeneration completes successfully after each regenerate action. • Newly added urgent task is included in schedule and prioritized appropriately. • Updated study limit (6 hours/day) is applied in the second regenerated schedule. • Existing study blocks are rebalanced to accommodate new task and settings. • Regenerated schedule persists after page refresh.
Priority	High
Pass/Fail Criteria	Pass: Regenerated schedule reflects both task and preference updates. Urgent task is scheduled with high urgency. New daily limit is correctly applied without data loss. Fail: Regeneration ignores changes. Urgent task is missing/not prioritized. Daily limit remains unchanged. Schedule data is lost or corrupted.

ID	TC-24
Item to Test & Description	Dashboard Statistics Accuracy: Verify dashboard correctly displays total tasks, overdue tasks, and total estimated hours, and updates after task changes.
Pre-Conditions	1. Active internet connection is available. 2. Backend is running. 3. Frontend is running. 4. User is logged in. 5. User has at least 4 tasks with mixed deadlines and statuses.
Test Steps	1. Navigate to "Dashboard" page. 2. Locate statistics section and record displayed values: a. Total Tasks b. Overdue Tasks c. Total Hours Needed (estimated) 3. Navigate to "Tasks" page. 4. Manually compute expected values: a. Count all active tasks for Total Tasks metric (based on app rule). b. Count tasks with deadline earlier than current date/time for Overdue Tasks. c. Sum Estimated Hours for applicable tasks (based on app rule). 5. Return to Dashboard page. 6. Compare dashboard metrics to manual calculations from step 4. 7. Create a new task with: a. Title: Dashboard Metric Test Task b. Course: select any existing course c. Deadline: 2 days from now d. Estimated Hours: 1.5 e. Priority: Medium 8. Save task and verify success message appears. 9. Return to Dashboard and verify: a. Total Tasks increments by 1. b. Total Hours Needed increases by 1.5 (or according to rounding/display rules). 10. Mark one active task as Complete on Tasks page. 11. Return to Dashboard and verify metrics update according to app rules (for example active total decreases by 1). 12. Refresh Dashboard and verify all displayed metrics remain consistent.
Expected Results	• Dashboard displays correct total task count • Dashboard displays correct overdue task count (2) • Dashboard displays correct total hours (6.5 hours) • Statistics match manual count from Tasks page • Statistics update when tasks are added • Statistics update when tasks are marked complete
Priority	High
Pass/Fail Criteria	Pass: All dashboard statistics are accurate, match manual verification, and update correctly after add/complete actions. Fail: Any dashboard metric is incorrect, does not update, or becomes inconsistent with task data.

ID	TC-25

Item to Test & Description	Dashboard Quick Actions Navigation: Verify Quick Actions buttons (Add Task, Add Course, View Schedule) open the correct destination and function without errors.
Pre-Conditions	1. Active internet connection is available. 2. Backend is running. 3. Frontend is running. 4. User is logged in. 5. Dashboard page loads successfully.
Test Steps	1. Navigate to “Dashboard” page. 2. Locate Quick Actions section. 3. Verify all three buttons are present and labeled: a. Add Task b. Add Course c. View Schedule 4. Click Add Task. 5. Verify “Tasks” page opens. 6. Verify task input fields are visible and editable. 7. Close/cancel without saving and return to “Dashboard”. 8. Click Add Course. 9. Verify “Course” page opens. 10. Verify course input fields are visible and editable. 11. Close/cancel without saving and return to “Dashboard”. 12. Click View Schedule. 13. Verify user is navigated to “Schedule” page. 14. Verify schedule calendar component loads successfully. 15. Use browser Back button to return to Dashboard and repeat each quick action once to confirm consistency.
Expected Results	• All quick action buttons are visible and correctly labeled. • Add Task opens task creation flow correctly. • Add Course opens course creation flow correctly. • View Schedule navigates to schedule page and loads calendar. • No routing errors, blank pages, or broken modals occur. • Repeated clicks produce consistent behavior. • Quick actions save time for common tasks
Priority	Medium
Pass/Fail Criteria	Pass: All quick actions route to the correct destination, required UI loads correctly, and actions work consistently with no errors. Fail: Any button is missing, routes incorrectly, fails to open target UI, or causes runtime/navigation errors.

ID	TC-26
Item to Test & Description	Preference Quiz Persistence and Usage: Verify quiz responses are saved to Supabase and are applied during schedule generation.
Pre-Conditions	1. Active internet connection is available. 2. Backend is running. 3. Frontend is running. 4. User is logged in. 5. User has at least 2 courses and 3 tasks available for schedule generation. 6. No locked/manual overrides exist in schedule that would hide preference effects.
Test Steps	1. Navigate to “Profile” page.

	2. Open Preference Quiz. 3. Enter quiz responses exactly as follows: a. Preferred Work Time: Morning (6:00 AM-12:00 PM) b. Preferred Work Days: Monday-Friday c. Break Timings: Full Immersion (90+ minutes) d. Assignment Plan: Balanced 4. Click Submit. 5. Verify a success message appears. 6. Refresh "Profile" page. 7. Re-open Preference Quiz and verify all saved values still match step 3. 8. Navigate to "Schedule" page. 9. Click "Generate Schedule". 10. Wait until generation completes (no loading spinner). 11. Inspect generated blocks and verify: a. Study block lengths are near 90 minutes where feasible. b. Break gaps between consecutive blocks are near 30 minutes where feasible. c. Morning slots are used preferentially for major work blocks when slots are available. 12. Regenerate schedule once more after changing one quiz value (for example Break Timings from 90 to 60). 13. Verify newly generated schedule reflects updated quiz value.
Expected Results	• Quiz responses saved successfully to database • Quiz selections persist after page refresh • Quiz preferences actively used in schedule generation • Schedule generation respects all quiz selections • Block sizes match quiz preferences (90 min) • Break durations match quiz preferences (15 min) • Peak time preferences applied (morning tasks prioritized) • Quiz can be re-taken and changes apply to future schedules
Priority	High
Pass/Fail Criteria	Pass: Quiz values are saved and persisted, and generated schedules reflect those values consistently. Fail: Quiz values are not saved/persisted, or schedule output ignores saved quiz settings.

ID	TC-27
Item to Test & Description	Schedule View Consistency: Verify Day, Week, Month, and Year views display the same underlying courses/tasks/study blocks correctly.
Pre-Conditions	1. Active internet connection is available. 2. Backend is running. 3. Frontend is running. 4. User is logged in. 5. A generated schedule exists with: a. At least 1 course block. b. At least 3 study blocks across different days. c. At least 1 break block.
Test Steps	1. Navigate to "Schedule" page. 2. Click "Day View" to select day view option. 3. Verify calendar shows single day with hourly grid. 4. Verify course meeting times displayed.

	5. Verify task time blocks displayed with task titles. 6. Verify correct start and end times shown. 7. Verify break blocks appear between study sessions. 8. Navigate to next day (verify blocks update). 9. Verify blocks don't repeat across days. 10. Click “Week View” option. 11. Verify calendar shows 7 days (Mon-Sun or Sun-Sat). 12. Verify all blocks visible for entire week. 13. Verify correct day alignment. 14. Verify courses appear on correct meeting days. 15. Verify tasks appear on correct days. 16. Click next week arrow to navigate forward. 17. Verify calendar updates to show next week. 18. Click “Month View” option. 19. Verify full month calendar displayed. 20. Verify blocks shown as indicators on each day. 21. Verify task/course indicators visible. 22. Verify current day highlighted. 23. Click month navigation arrows to move forward/backward.
Expected Results	• Day view shows accurate single-day schedule • Week view shows all 7 days with proper alignment • Month view shows monthly overview with indicators • Year view shows annual overview • All views show same underlying schedule data • Times and dates consistent across all views • No missing blocks or data when switching views • Course and task information visible in all views • Navigation works smoothly between views
Priority	High
Pass/Fail Criteria	Pass: All views are data-consistent, time-accurate, and navigation functions correctly with no missing/duplicate entries. Fail: Any view shows inconsistent data, wrong times/dates, missing entries, duplicate entries, or broken navigation.

5.4 System Test Cases

ID	TC-28
Item to Test & Description	System End-to-End Validation: Verify a complete user workflow from login to schedule generation to task completion works correctly across all connected modules.
Pre-Conditions	1. Active internet connection is available. 2. Frontend is running. 3. Backend is running. 4. Supabase services are available. 5. User has a valid account and can log in. 6. User has at least one existing course before test start.
Test Steps	1. Navigate to “Login” page and log in with valid credentials. 2. Navigate to “Tasks” page and create a new task with: a. Title: End-to-End Test Task b. Associated Course: select any existing course c. Deadline: 2 days from now at 11:59 PM d. Estimated Hours: 2

	e. Priority: High 3. Save task and verify success message appears. 4. Navigate to "Profile" page and complete Preference Quiz with: a. Preferred Work Time: Evening b. Preferred Work Days: Monday-Friday c. Break Timings: 60 minutes d. Assignment Plan: Balanced 5. Save preferences and verify success message appears. 6. Navigate to Schedule page and click Generate Schedule. 7. Wait for generation to complete (max 10 seconds). 8. Verify generated schedule includes the new End-to-End Test Task. 9. Verify generated blocks do not overlap existing course meeting times. 10. Navigate to Dashboard and verify task statistics include newly created task. 11. Navigate to Tasks page and mark End-to-End Test Task as Completed. 12. Return to Dashboard and verify pending/completed counts update correctly. 13. Refresh browser page. 14. Verify all data persists after refresh: a. Task still marked Completed b. Generated schedule blocks still visible c. Dashboard metrics remain correct 15. Log out. 16. Log back in and verify persisted data is still correct.
Expected Results	• User can complete full workflow without errors across Auth, Tasks, Preferences, Schedule, and Dashboard modules. • New task is generated into schedule and respects constraints. • Dashboard metrics update after task completion. • Data persists across refresh and re-login sessions. • No data mismatch appears between pages.
Priority	High
Pass/Fail Criteria	Pass: Entire end-to-end flow executes successfully, all modules remain synchronized, and data persists correctly after refresh and re-login. Fail: Any module fails in workflow, schedule/task/dashboard become inconsistent, or data is lost after refresh/session restart.

ID	TC-29
Item to Test & Description	First-Time User End-to-End Setup: Verify a new user can sign up, create initial data, generate a schedule, and see a usable dashboard in one continuous flow.
Pre-Conditions	1. Active internet connection is available. 2. Frontend, backend, and Supabase services are running. 3. Test email is not already registered.
Test Steps	1. Navigate to "Register/Login" page. 2. Register a new account with: a. First Name: New b. Last Name: Student c. Email: newstudent_test@wayne.edu d. Password: TestPass123! e. Confirm Password: TestPass123! 3. Verify email using Supabase verification link. 4. Log in with new credentials.

	5. Navigate to “Courses” page and create one course with valid days and times. 6. Navigate to “Tasks” page and create two tasks linked to that course with deadlines in the same week. 7. Navigate to “Profile” page and complete Preference Quiz with valid values. 8. Navigate to “Schedule” page and click “Generate Schedule”. 9. Verify generated study blocks appear. 10. Navigate to “Dashboard” page and verify task stats and pending list are populated. 11. Refresh browser and verify all created data persists.
Expected Results	• User completes onboarding flow without errors. • Created course/tasks/preferences are used by schedule generation. • Dashboard reflects generated and stored data correctly. • Data persists after refresh.
Priority	High
Pass/Fail Criteria	Pass: Entire onboarding workflow succeeds from account creation to generated schedule and dashboard visibility with persistent data. Fail: Any major step fails (registration, login, create data, generate schedule, dashboard update, or persistence).

ID	TC-30
Item to Test & Description	Regeneration Ripple Test: Verify editing an existing course and task updates regenerated schedule and all related displays consistently.
Pre-Conditions	1. Active internet connection is available. 2. Frontend, backend, and Supabase are running. 3. User is logged in. 4. User has at least 2 courses, 3 active tasks, and an existing generated schedule.
Test Steps	1. Navigate to “Courses” page and edit one course time block (shift by +1 hour). 2. Save course changes and confirm success message. 3. Navigate to “Tasks” page and edit one task: a. Deadline: move earlier by 1 day b. Priority: set to High c. Estimated Hours: increase by 1 hour 4. Save task changes and confirm success message. 5. Navigate to “Schedule” page and click “Generate Schedule”. 6. Verify updated task appears earlier in schedule than before. 7. Verify no generated study block overlaps with edited course time. 8. Navigate to Day, Week, and Month views and confirm updates are consistent across views. 9. Navigate to “Dashboard” and verify task counts/hours reflect edited values. 10. Refresh page and verify updates persist.
Expected Results	• Edited course/task values propagate to schedule generation immediately. • Regenerated schedule honors new deadlines/priority and avoids course conflicts. • Calendar views and dashboard show consistent updated data. • Changes persist after refresh.
Priority	High
Pass/Fail Criteria	Pass:

	All dependent modules (Schedule, Calendar views, Dashboard) reflect edited data accurately after regeneration. Fail: Any module shows stale data, conflicting blocks, or inconsistent values.

ID	TC-31
Item to Test & Description	Session Persistence and Recovery: Verify user session and data continuity after browser refresh and re-login within an active planning workflow.
Pre-Conditions	1. Active internet connection is available. 2. Frontend, backend, and Supabase are running. 3. User is logged in with existing courses, tasks, and generated schedule.
Test Steps	1. On Dashboard, note current task counts and one visible pending task name. 2. Navigate to Schedule page and note one generated study block title/time. 3. Refresh browser page. 4. Verify user remains authenticated and current page reloads correctly. 5. Verify noted dashboard values and schedule block remain unchanged. 6. Log out. 7. Log back in with valid credentials. 8. Verify user is redirected to authenticated page. 9. Re-open Dashboard and Schedule pages. 10. Verify previously noted task counts and schedule block still match pre-logout state.
Expected Results	• Refresh does not corrupt session or user data. • Logout/login cycle restores access to the same persisted data. • No unexpected data loss, duplication, or state mismatch occurs.
Priority	Medium
Pass/Fail Criteria	Pass: User data and schedule remain consistent through refresh and re-login, with correct authenticated routing. Fail: Session breaks unexpectedly, data disappears/duplicates, or values mismatch after re-login.

ID	TC-32
Item to Test & Description	Canvas-to-Planner System Flow: Verify assignments synced from Canvas extension are stored, displayed, and schedulable in the full 11Planner system.
Pre-Conditions	1. Active internet connection is available. 2. Frontend, backend, and Supabase are running. 3. User is logged into 11Planner and Canvas in same browser. 4. Chrome extension is installed and enabled. 5. Canvas course page contains at least 3 assignments not yet present in 11Planner.
Test Steps	1. Open Canvas course assignments page and verify 3 target assignments are visible. 2. Click extension icon and trigger assignment sync. 3. Wait for sync completion message. 4. Open 11Planner Tasks page. 5. Verify synced assignments appear as tasks with correct titles and due dates. 6. Navigate to Schedule page and click Generate Schedule. 7. Verify synced tasks receive generated study blocks. 8. Verify generated blocks appear in Week view and persist after refresh.

	9. Check Dashboard and verify task totals include synced assignments.
Expected Results	• Canvas assignments are imported into 11Planner successfully. • Imported tasks are stored and visible in Tasks and Dashboard. • Schedule generation includes imported tasks without errors. • Data persists after refresh.
Priority	Medium
Pass/Fail Criteria	Pass: Full chain (Extension -> Backend -> Tasks -> Schedule -> Dashboard) works with accurate synced data and persistence. Fail: Sync fails, imported data is missing/incorrect, tasks are unschedulable, or data is not persisted.

5.5 Acceptance Test Cases

ID	TC-33
Item to Test & Description	Core Planning Workflow: Verify a student can complete the full primary flow (login, add course, add tasks, set preferences, generate schedule, verify dashboard) without outside help.
Pre-Conditions	1. Active internet connection is available. 2. Frontend, backend, and Supabase services are running. 3. Test user account exists and is verified: a. Email: testuser@wayne.edu b. Password: TestPass123! 4. Test user starts logged out. 5. Test user has no generated schedule for current week before starting (clear baseline).
Test Steps	1. Navigate to “Register/Login” page. 2. Enter credentials: a. Email: testuser@wayne.edu b. Password: TestPass123! 3. Click Log In. 4. Verify redirect to “Profile” page within 3 seconds. 5. Navigate to “Courses” page and click “New Course”. 6. Enter course fields exactly: a. Course Name: Linguistics 101 b. Course Code: LING 101 c. Course Days: Monday, Wednesday d. Start Time: 10:00 AM e. End Time: 11:15 AM 7. Click “Create Course” and verify success message appears. 8. Navigate to “Tasks” page and click “New Task” to add Task 1: a. Task Title: Read Chapter 3 b. Course: LING 101 c. Deadline: 2 days from now, 08:00 PM d. Estimated Hours: 2 e. Priority: High 9. Click “Create Task” to add Task 1 and verify success message appears. 10. Click “New Task” again and add Task 2: a. Task Title: Phonetics Worksheet b. Course: LING 101 c. Deadline: 4 days from now, 09:00 PM d. Estimated Hours: 3 e. Priority: Medium 11. Click “Create Task” to add Task 2 and verify success message appears.

	12. Navigate to “Profile” page and select Preference Quiz, then set: a. Preferred Work Time: Evening b. Preferred Work Days: Monday-Friday c. Break Timings: 60 minutes d. Assignment Plan: Balanced 13. Submit quiz and verify success message appears. 14. Navigate to “Schedule” page. 15. Click “Generate Schedule” once. 16. Wait until loading spinner disappears (max 10 seconds). 17. Verify at least one block exists for Task 1 and Task 2. 18. Verify no block overlaps Monday/Wednesday 10:00-11:15 AM class time. 19. Navigate to “Dashboard” page. 20. Verify dashboard shows updated task totals and pending tasks list includes both tasks.
Expected Results	• User completes entire workflow without blockers. • Course and tasks are saved and visible in relevant pages. • Preferences are saved and applied during generation. • Generated schedule includes both tasks with no class-time conflicts. • Dashboard reflects created tasks and generated planning state. • Tester reports flow is understandable without assistance.
Priority	High
Pass/Fail Criteria	Pass: All steps complete successfully in one session, generated schedule is valid, dashboard data is correct, and tester confirms workflow clarity. Fail: Any major step fails, generated schedule is invalid/missing, dashboard mismatches data, or tester needs assistance to complete core flow.

ID	TC-34
Item to Test & Description	Interface Clarity and Usability: Verify users can quickly understand what to do next and navigate key pages without confusion.
Pre-Conditions	1. User is logged in. 2. Dashboard, Tasks, Courses, Schedule, and Settings pages are accessible. 3. Test is run on desktop browser at standard zoom (100%).
Test Steps	1. Open “Dashboard” page and start a 5-second timer. 2. Without clicking anything, ask tester to identify the next recommended action. 3. Record whether tester identifies an action within 5 seconds (for example Add Task or Generate Schedule). 4. Ask tester to navigate to “Tasks” page using top/side navigation only (no URL typing). 5. Record time to reach “Tasks” page (target: under 5 seconds). 6. Ask tester to navigate from “Tasks” to “Schedule” page using navigation only. 7. Record time to reach “Schedule” page (target: under 5 seconds). 8. Ask tester to return to “Dashboard” page using navigation only. 9. In “Settings” page, switch theme: a. Light mode b. Dark mode 10. In each mode, verify readability: a. Header text readable b. Button labels readable c. Task cards readable

	11. Ask tester to rate clarity 1-5 for: a. Navigation clarity b. Information hierarchy c. Visual readability
Expected Results	• Tester identifies a clear next action within 5 seconds on Dashboard. • Tester navigates between major pages quickly and without confusion. • UI text remains readable in both Light and Dark modes. • Tester provides acceptable clarity ratings (target: 4/5 in all categories).
Priority	Medium
Pass/Fail Criteria	Pass: Tester completes navigation tasks without confusion, identifies actions quickly, and rates clarity/readability at acceptable levels. Fail: Tester cannot identify next steps quickly, navigation causes confusion, or readability issues are reported in normal use.

ID	TC-35
Item to Test & Description	Data Reliability and Trust: Verify user data persists through refresh and re-login and remains consistent across pages.
Pre-Conditions	1. User is logged in. 2. User has at least 2 existing tasks and 1 generated schedule. 3. Active internet connection is available.
Test Steps	1. On “Tasks” page, create a new task: a. Title: UAT Persistence Check b. Course: select any existing course c. Deadline: 3 days from now, 07:00 PM d. Estimated Hours: 1.5 e. Priority: Medium 2. Save task and verify success message appears. 3. Navigate to “Schedule” page and click “Generate Schedule”. 4. Verify at least one block appears for UAT Persistence Check. 5. Navigate to “Dashboard” page and record: a. Total Tasks value b. Overdue Tasks value c. Total Hours value 6. Refresh browser/current page. 7. Re-check task existence in “Tasks” page. 8. Re-check block existence in “Schedule” page. 9. Re-check “Dashboard” page values and compare to step 5. 10. Log out. 11. Log back in with same account. 12. Re-check the same task, schedule block, and dashboard metrics.
Expected Results	• Newly created task persists after refresh and re-login. • Generated schedule block persists after refresh and re-login. • Dashboard statistics remain consistent with saved data. • No missing/duplicated data appears after session restart.
Priority	High
Pass/Fail Criteria	Pass: Data remains consistent across refresh and re-login with no loss or mismatch. Fail: Any data loss, duplication, or cross-page inconsistency occurs after refresh or re-login.

ID	TC-36
Item to Test & Description	Real-World Weekly Planning: Verify the app supports a realistic weekly planning session from setup to adjustment.
Pre-Conditions	1. User is logged in. 2. User has at least: a. 3 courses with meeting times b. 5 active tasks with mixed deadlines and priorities 3. Preference Quiz is completed. 4. Schedule generation is available.
Test Steps	1. Navigate to “Tasks” page and verify 5 active tasks are visible. 2. Confirm deadlines include at least one due within 48 hours and one due after 5+ days. 3. Navigate to “Schedule” page and click “Generate Schedule”. 4. Verify all 5 tasks receive at least one scheduled block. 5. Switch to Day view and verify today’s blocks are visible with times. 6. Switch to Week view and verify all study blocks are spread across the week. 7. Switch to Month view and verify schedule indicators appear on relevant dates. 8. Return to “Tasks” page and mark one urgent task as Completed. 9. Return to “Schedule” page and click “Generate Schedule”. 10. Verify completed task is removed from future pending study allocation. 11. Verify remaining tasks are rebalanced into available time. 12. Navigate to “Dashboard” page and verify pending/completed metrics reflect change.
Expected Results	• System supports full weekly planning and replanning in one session. • Generated schedule covers all active tasks appropriately. • Calendar views consistently represent the same underlying schedule data. • Completion status updates trigger meaningful schedule adjustments. • Dashboard reflects updated workload state accurately.
Priority	Medium
Pass/Fail Criteria	Pass: Tester can successfully plan, view, and adjust a realistic weekly workload with accurate updates across Tasks, Schedule, and Dashboard. Fail: Workflow breaks under realistic use, updates are inconsistent, or tester rejects practical usefulness.

5.6 Test Report

Test Case	Test Executor	Pass/Fail	Date	Additional Notes
TC-01	Suganeshwara, Aaraiz	Pass	03/24/2026	Account created successfully, verification email received within 1 minute, login worked after verification.
TC-02	Suganeshwara, Aaraiz	Pass	03/24/2026	Valid credentials created persistent session; protected routes remained accessible after refresh.
TC-03	Suganeshwara, Aaraiz	Pass	03/24/2026	Course created with all fields, persisted after refresh, and appeared in task course dropdown.
TC-04	Suganeshwara, Aaraiz			

TC-05	Suganeshwara, Aaraiz			
TC-06	Suganeshwara, Aaraiz			
TC-07	Suganeshwara, Ayham			
TC-08	Suganeshwara, Ayham			
TC-09	Suganeshwara, Ayham			
TC-10	Suganeshwara, Ayham			
TC-11	Suganeshwara, Ayham			
TC-12	Suganeshwara, Ayham			
TC-13	Aaraiz, Tamanna			
TC-14	Aaraiz, Tamanna			
TC-15	Aaraiz, Tamanna			
TC-16	Aaraiz, Tamanna			
TC-17	Aaraiz, Tamanna			
TC-18	Aaraiz, Tamanna			
TC-19	Ayham, Tamanna			
TC-20	Suganeshwara, Aaraiz, Ayham			
TC-21	Suganeshwara, Aaraiz, Ayham			
TC-22	Suganeshwara, Aaraiz, Ayham			
TC-23	Suganeshwara, Aaraiz, Ayham			
TC-24	Suganeshwara, Aaraiz, Ayham			
TC-25	Ayham, Tamanna			
TC-26	Ayham, Tamanna			
TC-27	Tamanna, Ayham			
TC-28	Tamanna, Ayham			
TC-29	Tamanna, Ayham			
TC-30	Tamanna, Ayham			
TC-31	Tamanna, Ayham			
TC-32	Suganeshwara, Aaraiz, Ayham, Tamanna			
TC-33	Suganeshwara, Aaraiz, Ayham, Tamanna			
TC-34	Suganeshwara, Aaraiz, Ayham, Tamanna			
TC-35	Suganeshwara, Aaraiz, Ayham, Tamanna			
TC-36	Ayham, Tamanna			

6. Test Schedule

The testing process for 11Planner will follow a structured, phased timeline to ensure that all layers of the application, from the isolated backend algorithms to the fully integrated frontend interfaces, are systematically evaluated. Unlike a traditional waterfall schedule, we allocate dedicated time for regression testing and bug fixing prior to final acceptance.

Below is the detailed test execution schedule:

Date Range	**Target Test Cases**	**Test Executor**	**Test Type**	**Focus Area / Description**
03/24 - 03/28	TC-01 to TC-06	Suganeshwara, Aaraiz	Functional	Authentication, course/task create + validation, core CRUD behavior.
03/27 - 04/01	TC-07 to TC-12	Suganeshwara, Ayham	Functional	Schedule generation logic, prioritization, constraints, preference effects, course edit propagation.
03/30 - 04/03	TC-13 to TC-18	Aaraiz, Tamanna	Functional	Delete/edit propagation, syllabus parsing, Canvas sync, timer controls.

04/02 - 04/06	TC-19, TC-25, TC-26, TC-30, TC-33, TC-34, TC-36	Ayham, Tamanna	Non-Functional	Performance, persistence, usability/clarity, security isolation, cross-view consistency.
04/05 - 04/10	TC-20 to TC-24	Suganeshwara, Aaraiz, Ayham	Integration	Dashboard-task sync, full schedule workflow, regeneration correctness, cross-module consistency.
04/09 - 04/13	TC-27 to TC-31	Tamanna, Ayham	System	End-to-end flows: login -> create/import -> plan -> regenerate -> track -> persist.
04/14 - 04/16	Failed/Blocked Cases from TC-01 to TC-36	Suganeshwara, Aaraiz, Ayham, Tamanna	Regression	Retest failed cases after fixes and verify no regressions.
04/17 - 04/18	TC-32 to TC-35	Suganeshwara, Aaraiz, Ayham, Tamanna	Acceptance	Final user-facing validation and project sign-off readiness.

Schedule Notes:
Parallel Execution: You will notice some dates overlap (e.g., Frontend and Backend Unit testing). Because our architecture separates concerns, these unit tests can be executed simultaneously without blocking each other.
Acceptance Testing: This is the terminal phase of the project where the client (represented by our Professor/GTA) reviews our implementation. Approval at this stage signifies the project is ready for final submission.

7. Test Environment

To accurately validate the 11Planner system, our testing environment must mirror the eventual production use-cases as closely as possible while providing developers the tools necessary to debug code locally. Because our application operates as a full-stack platform requiring both backend AI processing and a media-rich frontend, testing requires specialized software versions and explicit setup configurations.

7.1 Hardware Requirements

While 11Planner is a web-based application requiring no specialized test servers, developers and QA personnel running the local build environments must meet the following hardware baselines:

- CPU: Intel i5 / AMD Ryzen 5 or better (4+ cores recommended for concurrent compilation).
- RAM: Minimum 8GB (16GB highly recommended to prevent bottlenecks when running Node.js and Python simultaneously).
- Disk Space: 20GB free space to accommodate node_modules, standard Python libraries, and generated test/coverage artifacts.
- Monitors: At least one 1920x1080 resolution monitor to ensure UI components and responsive React grids render accurately during manual testing.
- Network: Stable broadband internet connection required to communicate with Supabase, OpenAI, and Canvas APIs.

7.2 Software & Dependency Requirements

Backend Technologies:

- Python 3.8 or higher
- FastAPI web framework & Uvicorn ASGI server
- python-dotenv (for secure environment variable handling)
- openai==1.58.0 (for AI schedule optimization)
- pypdf==5.4.0 (for syllabus PDF parsing capabilities)

- supabase==2.15.3 (for cloud database interactions)

Frontend Technologies:

- Node.js 18 or higher
- npm or yarn package manager
- Next.js App Router framework
- React & TypeScript implementation
- Tailwind CSS

Testing Toolchain:

- Python: pytest, unittest, pytest-cov
- JavaScript: Jest, React Testing Library
- API Validation: Postman
- Performance: k6 or Artillery
- Supported Browsers: Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge (latest stable builds required).

Third-Party Services:

- Supabase: Cloud database instance.
- OpenAI API: Requires active test-tier API keys.
- Canvas API: Access tokens required for external assignment synchronization integration.

7.3 Environmental Conditions

To ensure absolute consistency across testers, the local testing environment must be initialized using the precise steps outlined below.

Backend Setup & Execution:

```
cd backend
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt

# Create .env file from template
copy .env.example .env
# Testers MUST edit .env to include the active test-tier OpenAI API key.

# Start backend server
uvicorn app.main:app --reload
# Backend will successfully initialize at http://127.0.0.1:8000
```

Frontend Setup & Execution:

```
cd frontend
npm install
npm run dev
# Frontend will successfully compile and route to http://localhost:3000
```

Database Configuration Notes:

- 11Planner utilizes Supabase cloud architecture; therefore, no localized SQL database initialization (like pg_ctl) is required on the tester's machine.
- Testers must verify that the SUPABASE_URL and SUPABASE_ANON_KEY are properly injected into the backend .env file prior to launching FastAPI.
- Prior to executing rigorous test cases, testers are instructed to inject mock JSON sample data to verify the read/write connection is established.

8. Test Entry and Exit Criteria

To enforce strict quality assurance, 11Planner operates under a rigid, gated testing methodology. Testing phases are not fluid; they represent strict architectural checkpoints. A cycle cannot officially launch until all entry prerequisites are locked, and code cannot proceed to deployment until every exit metric is flawlessly satisfied.

8.1 Entry Criteria

Testing for any project milestone will strictly commence only when the following foundational conditions are 100% verified across the environment:

- Absolute Code Freeze: The development phase is locked. Zero new feature commits or speculative code merges are permitted to bypass the testing team without explicit lead approval.
- Environment Verification: The full-stack local environment is actively running without exceptions (FastAPI backend active, Next.js frontend compiled and accessible without fatal warnings).
- Infrastructure Connectivity: All third-party pipelines must be confirmed live. Specifically, the secure OpenAI API token must be configured and responsive, and the Supabase staging database must be accessible to both read and write operations.
- Documentation Sign-off: The comprehensive Test Plan has received formal approval from the Project Manager / Professor.
- Test Case Architecture: All discrete test case scenarios are fully designed, peer-reviewed, and mapped definitively to the Requirements Traceability Matrix.
- Data Readiness: Mock test payloads, simulated Canvas API JSON returns, sample user accounts, and varying syllabus PDFs are pre-loaded and injected into the test ecosystem. All necessary testing tools (Pytest, Jest, Postman, Playwright) are installed and fully operational.

8.2 Exit Criteria

The testing phase is an absolute bottleneck designed to catch catastrophic failures before release. Testing will not be declared formally complete until these uncompromising benchmarks are hit:

- Zero Critical Defects: Every single “Critical” or “Blocking” severity defect—specifically any instance resulting in data corruption, schedule overlaps, or system crashes—has been resolved, and the patch has passed rigorous regression re-testing.
- 100% Priority Case Success: All test cases categorized as “High-Priority” operate exactly as intended without exception.
- Coverage Mandates: Automated backend validation (via pytest-cov) definitively proves the codebase exceeds the mandatory 80% code-coverage minimum threshold.
- Performance Baselines: Network and algorithmic latency metrics are confirmed. All primary API response times consistently meet the <500ms target limit under standard simulation loads.
- Security Validation: All user authentication flows are confirmed secure, sanitization logic is actively blocking injection vectors, and no API keys are exposed to the browser client or network payloads.
- Matrix Verification: The Traceability Matrix confirms 100% of all assigned project requirements have corresponding, executed test cases.

- Final Sign-off: A comprehensive Test Execution Report is drafted, finalized, and formally signed off by the team lead and project stakeholders.

9. Test Pass and Fail Criteria

To eliminate ambiguity during QA execution, the 11Planner team operates under explicit, binary pass/fail standards. A test either completely satisfies the established project requirement, or it is formally logged as a defect. There is no concept of a “partial pass”.

9.1 Test Pass Criteria

A test case officially PASSES only when the following strict conditions are met concurrently:

- Exact Outcome Match: The expected results outlined in the test design are achieved exactly as specified without deviation.
- Zero Unexpected Exceptions: No unexpected system warnings, browser console errors, or backend Python stack traces occur during execution.
- Data Integrity: Application data (such as newly created tasks or AI-generated calendar blocks) is correctly persisted to the Supabase database and reliably retrieved upon page refresh.
- UI Reliability: The Next.js user interface responds correctly, predictably, and fluidly to all mouse and keyboard inputs (e.g., drag-and-drop schedule adjustments).
- Performance Tolerances: All performance metrics (API network calls, rendering the media-heavy Focus Dashboard, schedule generation computations) fall comfortably within the acceptable target ranges.

9.2 Test Fail Criteria

A test case immediately FAILS and triggers a defect report if it encounters any of the following conditions:

- Outcome Deviation: The expected functional results are not achieved.
- System Hangs or Crashes: The application loop breaks, resulting in frontend freezes (infinite loading spinners), unresponsive API routes, or hard backend server termination.
- Data Loss or Corruption: Supabase data is silently lost, overwritten, or corrupted by poorly formatted REST payloads.
- Rendering Errors: Incorrect data is displayed to the user (e.g., the AI algorithm double-books a time slot, or a parsed syllabus displays the wrong due date).
- Security Breaches: Major security vulnerabilities are discovered, such as the ability to bypass route authorization or the accidental exposure of backend environment keys.
- Unacceptable Latency: System execution pushes the application heavily beyond acceptable performance thresholds (e.g., basic API responses exceeding the 500ms limit under normal loads).

10. Test Suspension and Resumption Criteria

Efficiency in QA requires knowing when it is a waste of resources to continue testing a broken build. If the core architecture of 11Planner is fundamentally compromised, testers will not blindly continue logging sequential failures. Instead, testing will be formally suspended until the root blocker is resolved.

10.1 Suspension Criteria

The testing phase will be immediately suspended if any of the following “show-stopping” conditions are encountered:

- Critical Execution Blocker: A P1/Critical defect prevents basic application initialization (e.g., the FastAPI backend crashes continuously on startup, or the Next.js frontend throws a fatal compilation error).

- Environment Unavailability: The test environment becomes fundamentally corrupted or disconnected (e.g., the Supabase staging database aggressively rejects all connections).
- Third-Party API Outage: Essential dependencies, explicitly the Canvas LMS or OpenAI integrations, experience widespread downtime and testing variables cannot be functionally mocked.
- Data Scarcity: No valid test data (mock syllabi, simulated accounts) is available to execute the planned test cases.
- Mass Cascading Failures: More than 50% of the critical application functionality triggers failure states due to a shared foundational bug (e.g., broken secure routing).

10.2 Suspension Process

If a suspension criterion is met, the team must strictly adhere to the following emergency protocol:

1. Documentation: The QA tester officially documents the exact reason for the suspension, capturing all relevant logs and environment states.
2. Notification: The Test Lead notifies all project stakeholders and developers that the testing pipeline is formally paused.
3. Ticket Generation: A "Blocking Defect" ticket is created in the GitHub Issues/Jira board detailing the specific critical failure.
4. Assignment: The project manager instantly assigns the ticket to a lead resolution owner alongside an expected turnaround timeline.
5. Execution Halt: All ongoing manual and automated test execution is paused to prevent the logging of false-positive failures.

10.3 Resumption Criteria

Once testing has been suspended, it cannot be arbitrarily restarted. Testing will only be officially resumed when the following criteria are validated:

- Blocker Patched: The development team successfully patches the blocking issue, merges the code to the test branch, and executes a successful isolated test.
- Environment Restored: The local test servers, .env variables, and the Supabase database connections are restored and stable.
- Data Availability Confirmed: Accurate mock data payloads are confirmed to be loaded and accessible.
- Core Stability Confirmed: A high-level smoke test confirms that at least 80% of critical legacy functionality is stable prior to resuming full-scale testing.

10.4 Resumption Process

Once the resumption criteria are met, testing will be officially reinstated via the following process:

1. Resolution Verification: The Test Lead formally verifies and closes the blocking ticket.
2. Regression Validation: Testers immediately re-run the specific test cases that originally triggered the suspension to ensure the patch was effective.
3. Report Documentation: The suspension event and subsequent resumption are historically logged within the master Test Execution Report.
4. Pipeline Continuation: The team proceeds with the execution of all remaining untested cases from the point of pause.

11. Test Design and Execution

11.1 Approach Selection: Hybrid Methodology

For the 11Planner project, we have explicitly chosen a Hybrid Testing Approach, utilizing both Manual and Automated testing to ensure complete coverage across our varied tech stack.

Manual Testing

- Use Cases: User authentication flows, validating generated schedules, navigating UI routing, and testing media playback on the Focus Dashboard.
- When Used: Necessary for UI/UX validation, analyzing complex user workflows, exploratory testing, and attempting to manually reproduce bugs flagged by end-users.
- Tools: Web browsers (Chrome/Firefox/Safari) and native Browser DevTools.
- Advantage: Unbeatable for evaluating human-centric elements like aesthetic layout, responsive CSS behaviors, and overall application intuitiveness that machines cannot easily judge.

Automated Testing

- Use Cases: Verifying AI schedule-blocking math, regression testing during pull requests, API endpoint validation, and database load performance.
- When Used: Implemented for highly repetitive tests, strict data-type validation (Pydantic schemas), and maintaining consistent performance metrics.
- Tools: pytest (backend Python logic), Jest (frontend React state), and Postman scripting (automated API polling).
- Advantage: Offers lightning-fast execution times, absolute consistency, and ensures that legacy features do not break when new codebase updates are merged.

11.2 Test Case Structure

To maintain testing consistency and traceability, every authored Test Case within the design plan will strictly follow this standardized architecture:

- Test ID and Title: (e.g., TC-01: Valid Outline Upload)
- Component/Feature: (e.g., Syllabus Parser)
- Priority Level: Ranging from P0 (Critical/Blocker) to P3 (Low/Cosmetic).
- Pre-conditions: Requisite environment state before execution (e.g., “User must be logged in”).
- Test Steps: Detailed, sequential instructions for the tester to follow.
- Test Data: Specific inputs required (e.g., sample_syllabus_bio101.pdf).
- Expected Results: The exact, anticipated outcome of the system.
- Actual Results: Documented strictly during the execution phase.
- Pass/Fail Status: Binary outcome based on the criteria outlined in Section 9.
- Comments/Evidence: Links to screenshots, video captures, or terminal log dumps.

11.3 Test Execution Plan

The execution of our designed tests will follow a strict, sequential pipeline mirroring our software lifecycle to trap defects as early as possible.

Phase 1: Unit Testing (Days 1–2)

- Test individual Python functions, Pydantic models, and React components in total isolation.
- Heavily mock external AI and Canvas dependencies to prevent API token waste.

- Goal: Achieve and verify the mandatory 85%+ code coverage baseline.

Phase 2: Integration Testing (Days 2–3)

- Execute Postman scripts to validate FastAPI endpoint structures.
- Test JSON handshake integration between the Next.js frontend forms and the backend.
- Validate real read/write mutation operations against the localized Supabase staging database.

Phase 3: System Testing (Days 4–7)

- Manually test complete, end-to-end user workflows (Registration → Course Setup → Schedule Output).
- Verify the smooth transmission of data across all segregated components.
- Trigger frontend/backend error fallbacks to validate graceful exception handling.
- Execute baseline latency performance checks.

Phase 4: User Acceptance Testing [UAT] (Days 8–10)

- Introduce real-world, highly variable syllabus data.
- Conduct rigorous validation against actual academic workloads (simulating a student building a real mid-terms week schedule).
- Capture final project-requirement approvals from stakeholders/professors.

12. Test Data & Defect Management

12.1 Test Data Management

To ensure security and test reliability, no real student data or production database environments will be used during testing execution. All test data will be generated, collected, and managed systematically:

- **Data Isolation:** All manual and automated structural testing will occur on a cloned Staging instance of our Supabase database to ensure the production environment remains unpolluted.
- **Mock Payload Repository:** A dedicated /tests/mock_data/ directory will be maintained within the GitHub repository. To guarantee consistent testing inputs, this directory will house pre-configured JSON payloads (mimicking Canvas API responses) and a suite of varied syllabus files (PDFs, .docx, .txt) used to repeatedly hit the AI syllabus parser.
- **Documentation and Archiving:** As tests are executed in the staging environment, testers will record their outcomes within formal Execution Logs. Upon the conclusion of a testing block, the QA Lead will compile these logs into the final Requirements Traceability Matrix to prove that all data-handling thresholds were met.

12.2 Defect Management Process

When a QA tester or automated script identifies a failure, it will trigger a strict, standardized defect management lifecycle via our GitHub Issues tracking board.

12.2.1 Defect Logging

Any discovered bug must be immediately logged as a new ticket containing:

- A descriptive title and unique ID.
- The originating environment (e.g., “Safari browser, FastAPI Localhost”).
- The exact step-by-step actions required to reproduce the error.
- Supporting evidence, such as attached browser console screenshots or backend terminal stack traces.

12.2.2 Defect Triage and Prioritization

The Project Manager will review incoming defect logs and assign them a severity level to dictate development priority:

- **Critical (P0):** The defect prevents basic application initialization, destroys Supabase data, or crashes a server (e.g., the Next.js frontend entirely fails to render). Requires an immediate halt to testing and a hotfix.
- **High (P1):** A core feature is mathematically or functionally failing, but the application remains online (e.g., The schedule generator places two tasks in the same time block; the AI parser returns null dates). Fixed in order of discovery.
- **Medium (P2):** Application functionality works, but non-essential features are bugged or lagging (e.g., the Focus Dashboard video backgrounds stutter, or a button hover-state breaks). Fixes are planned for the next sprint.
- **Low (P3):** Minor cosmetic issues or typographical layout errors. Fixes are deployed when developers have spare bandwidth.

12.2.3 Defect Resolution Lifecycle

1. A developer claims (or is assigned) the defect ticket and analyzes the root cause.
2. The developer writes a patch on an isolated local Git branch and verifies the fix against the original failing test case.
3. The fix is merged into the staging branch, triggering a formal Regression Test by the QA tester.
4. If the regression test is entirely successful, the QA tester changes the ticket status to “Closed.”
5. If the defect behaviors persist, the ticket is marked as “Unresolved” and routed back to the developer for a secondary pass.

13. Risk Analysis

This section identifies potential risks associated with testing the 11Planner application and outlines mitigation strategies to minimize disruptions. Risks are categorized into Technical, Environmental, Human, and External Factors.

13.1 Technical Risks

Risk	Likelihood	Impact	Mitigation Strategy
OpenAI API key exposed in codebase	Medium	Critical	Implement strictly managed environment variables (.env). Ensure .gitignore is active. Conduct security code reviews for hardcoded secrets.
Schedule generation algorithm produces invalid schedules	Medium	High	Conduct extensive Python unit tests verifying mathematical edge cases. Implement proper locking to prevent data race conditions during concurrent requests.
Database data corruption or data loss	Low	Critical	Validate all JSON data models using Pydantic before persistence. Implement transaction testing and rollback safety protocols.
Performance degradation under load	Medium	High	Establish 500ms performance baselines. Conduct stress testing using Postman/Artillery. Implement Next.js caching where appropriate.
Memory leaks in long-running processes	Low	Medium	Utilize Chrome DevTools memory profiling specifically on the media-heavy Focus Dashboard components to optimize video rendering over long sessions.
Authentication token expiration dropping sessions	Low	Medium	Implement automated token refresh handling within the Supabase connection handlers to prevent users from losing unsaved schedule changes.

13.2 Environmental Risks

Risk	Likelihood	Impact	Mitigation Strategy
Cross-Site Scripting (XSS) or Injection Attacks	Medium	High	Test explicitly with OWASP Top 10 principles. Implement rigorous input validation on all manual task-entry forms and file uploads.
Testing on production database instance	Medium	Critical	Strictly separate environments. All testing must be pointed toward a dedicated Supabase staging database instance.
Network instability masking true endpoint latency	High	Medium	Execute all core performance benchmark testing over a stable, wired connection rather than testing on generic campus Wi-Fi.

13.3 Human Risks

Risk	Likelihood	Impact	Mitigation Strategy
Incomplete test coverage / missed edge cases	Medium	High	Mandate an 85% automated code coverage threshold via pytest-cov before any developer is allowed to merge code to the main branch.
Misinterpretation of test constraints	Low	High	Standardize all test cases with a rigid "Expected Results" criteria list. Require team members to cross-verify major structural fixes.
Testing resource/time exhaustion	Medium	Medium	Prioritize testing the core schedule logic and data integrity flows before dedicating manual testing hours to minor UI adjustments.

13.4 External Factors

Risk	Likelihood	Impact	Mitigation Strategy
Canvas API connection failures / Server Down	Low	High	Utilize Python unittest.mock to simulate endpoints for testing. Build robust error fallbacks routing users to manual entry.
OpenAI API Service Outage	Low	High	Mock external AI payloads. Ensure the system handles HTTP 503 errors gracefully without crashing the Uvicorn ASGI server.
Dependency updates breaking Next.js/FastAPI	Medium	High	Lock package versions strictly inside package.json and requirements.txt. Test library updates in an isolated branch before deployment.

13.5 Categorized Mitigation Execution Strategies

To actively execute the mitigations proposed in the matrices above, the development team will strictly adhere to the following organized technical guidelines:

13.5.1. Security Risk Mitigation

- Implement secure credential storage by masking all .env variables from the repository structure.
- Conduct manual security code reviews targeting explicit file-upload validation points.
- Test application forms utilizing OWASP Top 10 principles (specifically verifying sanitization limits logic).

13.5.2. Performance Risk Mitigation

- Establish strict performance baselines early in the phase (e.g., UI renderings taking under 3 seconds).
- Conduct load and stress testing using tools like Artillery, purposefully attempting to stretch the server capacity.
- Implement efficient database querying and Next.js static asset caching where appropriate.

13.5.3. Dependency Risk Mitigation

- Mock external APIs (via unittest.mock) to prevent testing delays when third-party servers undergo maintenance.
- Implement proper REST error handling (try/except closures) wrapping all outbound webhook requests.
- Create manual fallback mechanisms ensuring the application remains largely usable even if generative tools go offline.

13.5.4. Data Integrity Risk Mitigation

- Strictly validate all user data via Pydantic schema validation prior to any database persistence.
- Implement transaction handling so database modifications cleanly rollback if a multi-step upload fails midway.
- Routinely test Supabase backup and account recovery functionality over the testing lifecycle.

14. Roles and Responsibilities

The testing responsibilities of 11Planner will be clearly divided into distinct roles. There will be a total of 3 primary tester roles assigned across the team. These roles are designed to ensure that each aspect of our web application, from frontend validation to backend logic, is thoroughly managed and systematically executed.

Test Manager – Aaraiz Sohail
Responsibilities: Ensure that all tests have been run strictly according to the defined test schedule and that their results are recorded properly in the issue tracking system. They will also ensure that the Requirements Traceability Matrix is consistently kept up-to-date with the latest pass/fail test results, and they hold final sign-off authority before project submission.

Gray Box Testers – Suganeshwara Anand, Ayham Haq
Responsibilities: Perform the manual integration and end-to-end system test cases on the Next.js frontend (e.g., verifying Focus Dashboard rendering, file upload workflows, and schedule drag-and-drop mechanics). They are responsible for recording the results of these tests and communicating directly with the development team when tests fail, ensuring bugs are tracked, patched, and regression-tested again.

Unit Tester – Tamanna Gaur
Responsibilities: Create and maintain the automated unit tests specified in the testing plan. They will author the pytest and Jest testing suites targeting individual Python routing logic (like the schedule generator) and React components. They are responsible for running these test suites, mocking external dependencies (such as the OpenAI integration), and formally logging the code-coverage results.